Processor Architecture: Pipelined

by Alex 2500013032

2026.10.28

I. General Principles of Pipelining

顺序处理器 SEQ 在一个较长的时钟周期内完成一条指令的所有步骤：

$$\text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Execute} \rightarrow \text{Memory} \rightarrow \text{Write Back}$$

后一条指令必须等前一条指令完成整个周期，才能开始。

Unpipelined Execution



若把一条指令的工作切成多个阶段，就可以让不同指令同时占据不同阶段。

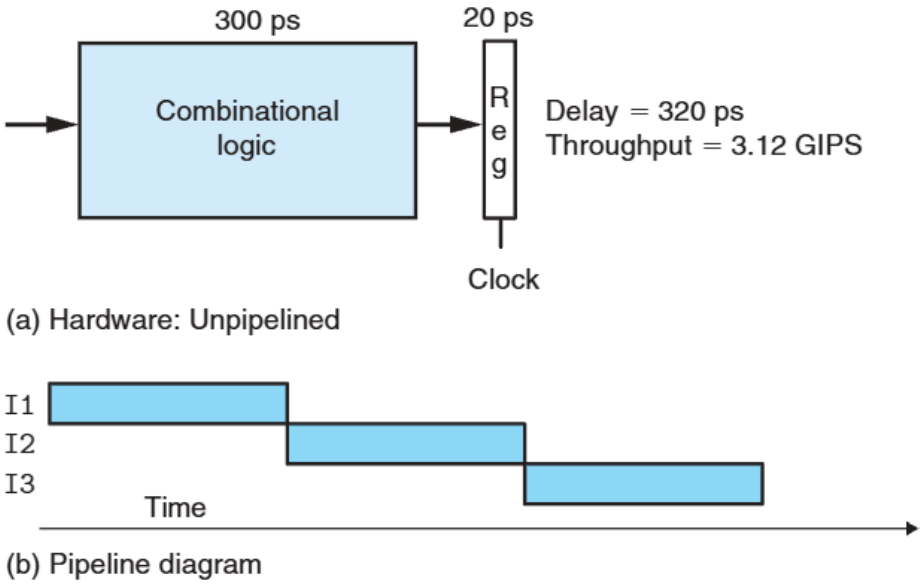
Five-stage Pipelined Execution

```
时间 ──────────────────────────────────────────────────────────►
I1:  [ F ][ D ][ E ][ M ][ W ]
I2:       [ F ][ D ][ E ][ M ][ W ]
I3:            [ F ][ D ][ E ][ M ][ W ]
I4:                 [ F ][ D ][ E ][ M ][ W ]
```

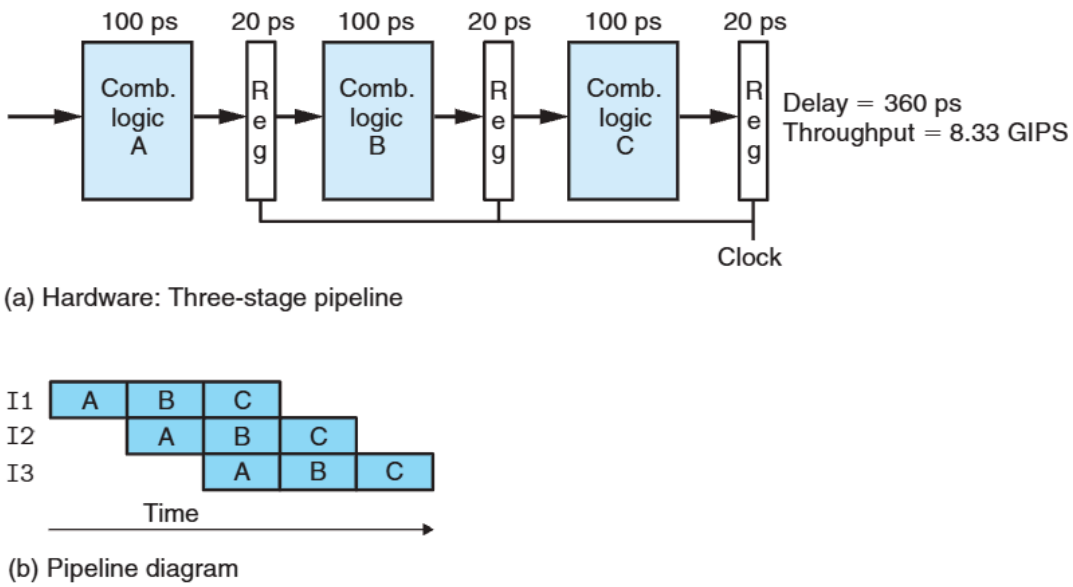
关键是让多条不同的指令在不同硬件阶段上重叠工作。流水线提高了系统的吞吐量（*Throughput*），但它可能会略微增加单条指令的端到端延迟（*Latency*）。

1. Computational Pipelines

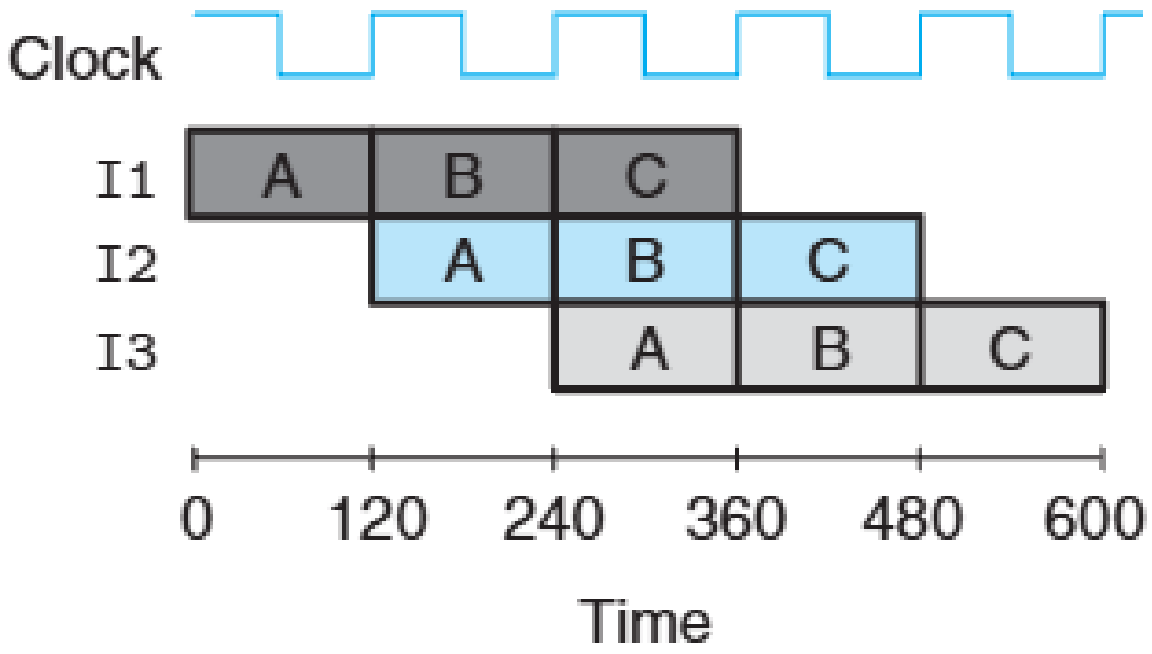
Unpipelined

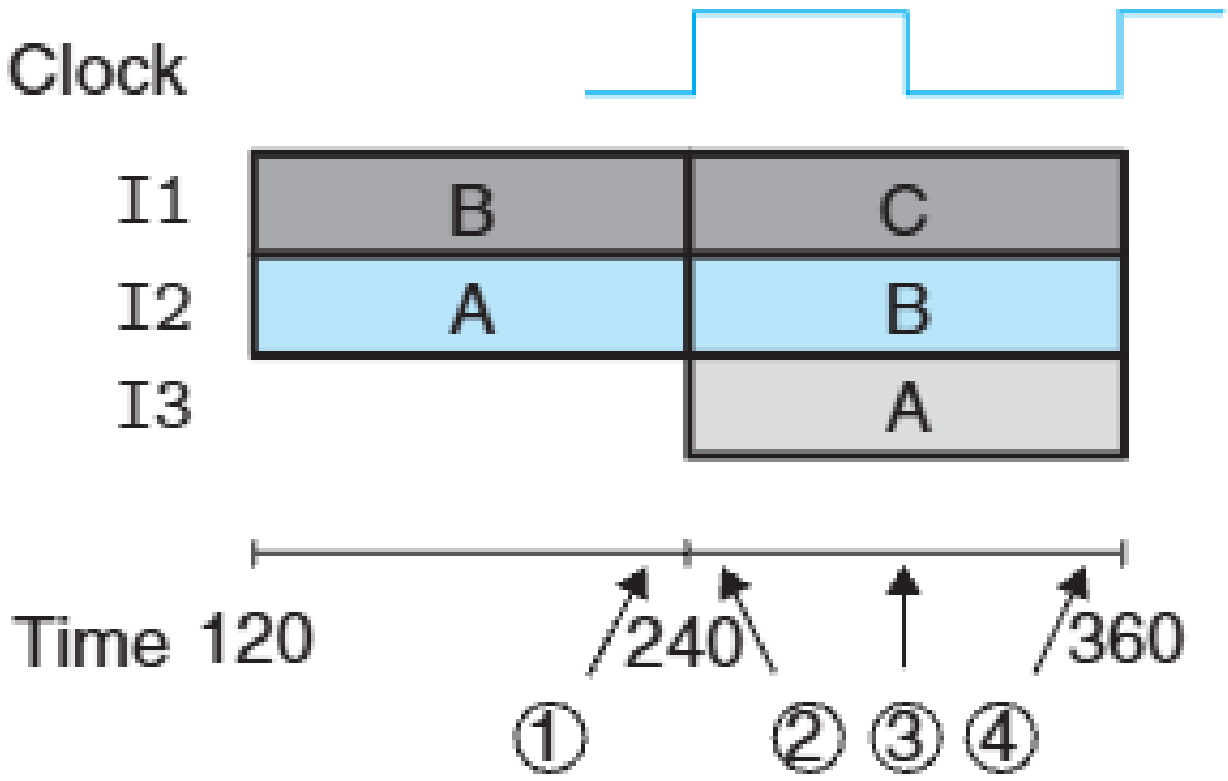


Pipelined

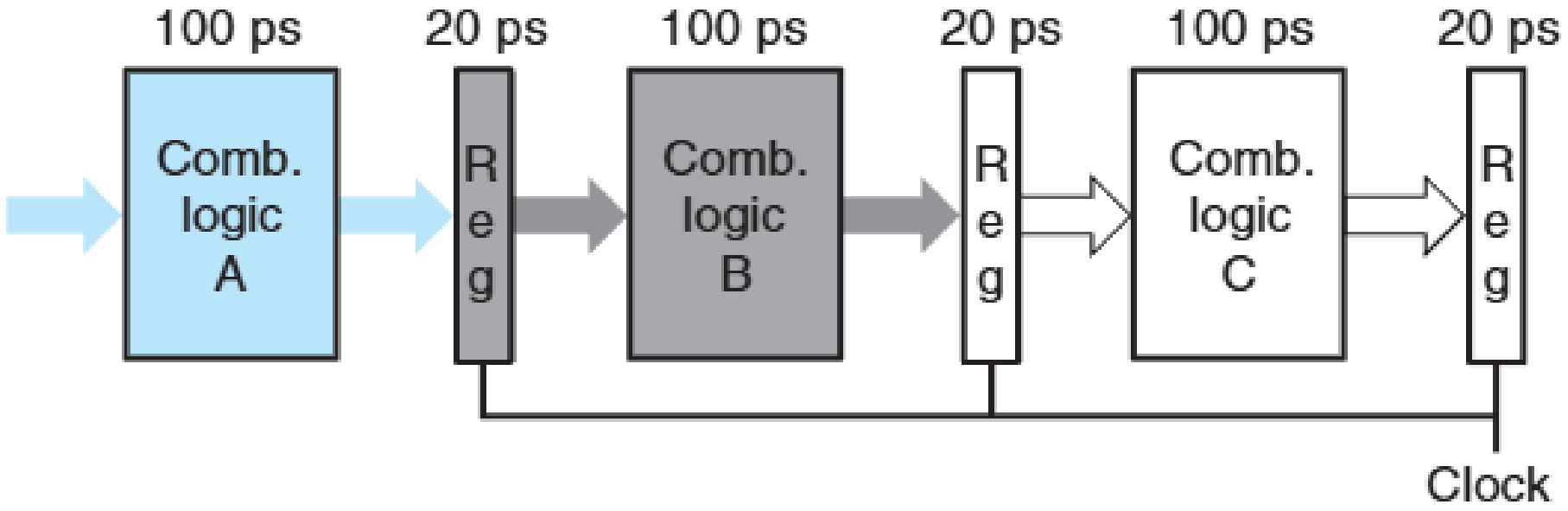


2. A Detailed Look at Pipeline Operation

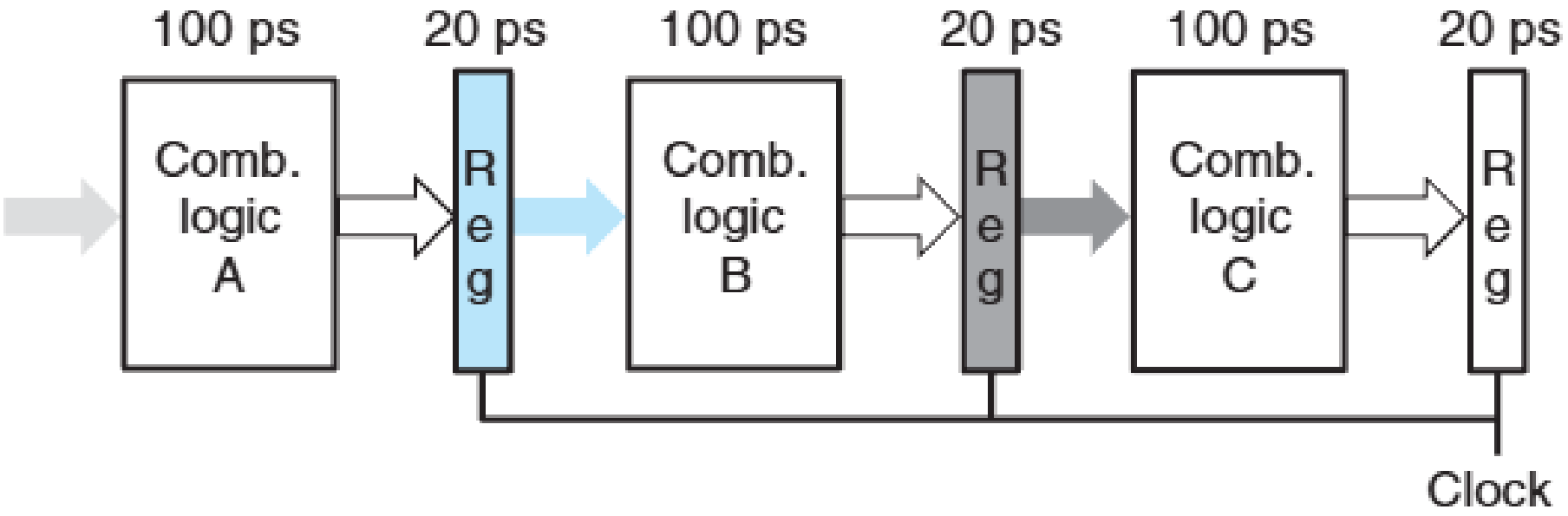



Clock
I1
I2
I3
B
C
A
B
A
Time 120
240
360
①
②
③
④

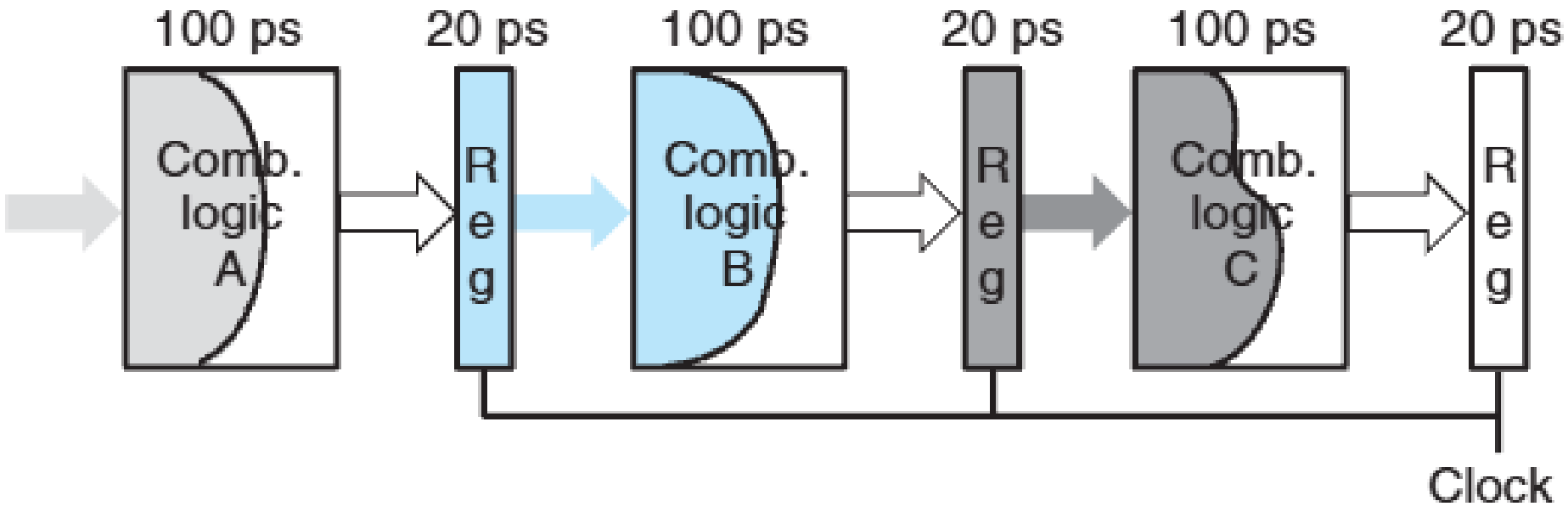
① Time = 239


100 ps
20 ps
100 ps
20 ps
100 ps
20 ps
Comb. logic A
Reg
Comb. logic B
Reg
Comb. logic C
Reg
Clock

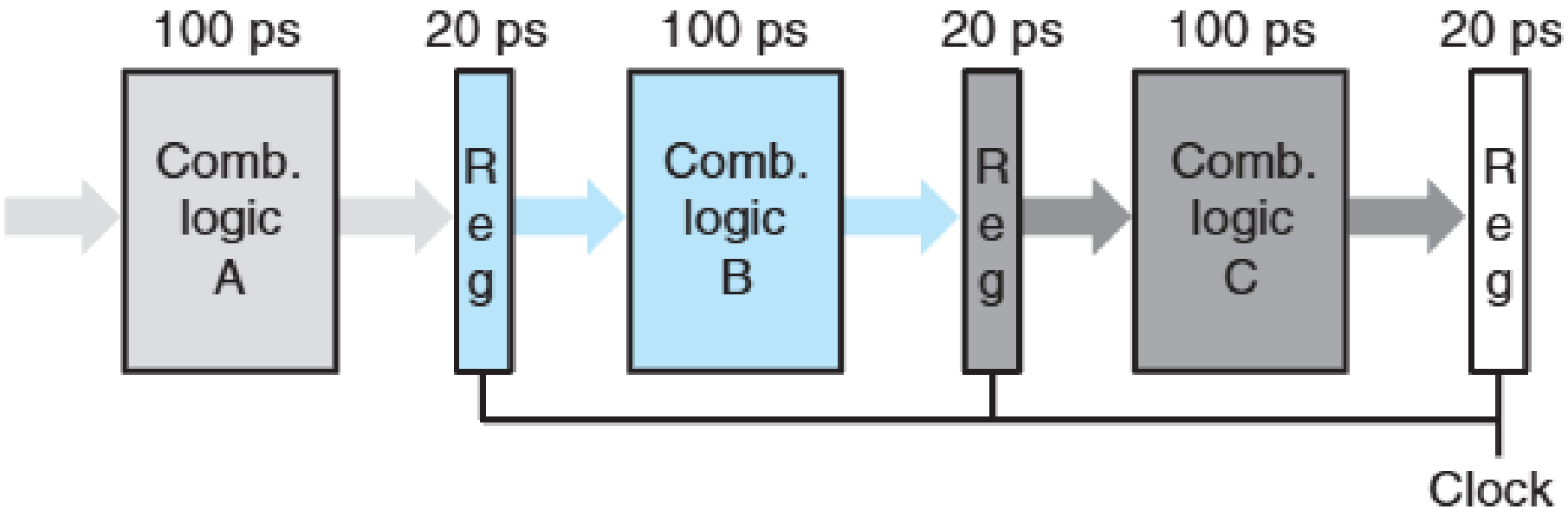
② Time = 241


100 ps
20 ps
100 ps
20 ps
100 ps
20 ps
Comb. logic A
Reg
Comb. logic B
Reg
Comb. logic C
Reg
Clock

③ Time = 300

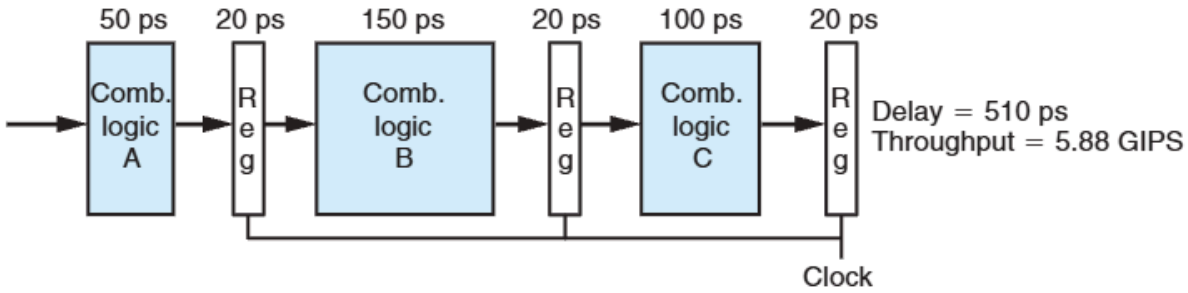

100 ps
20 ps
100 ps
20 ps
100 ps
20 ps
Comb. logic A
Reg
Comb. logic B
Reg
Comb. logic C
Reg
Clock

④ Time = 359

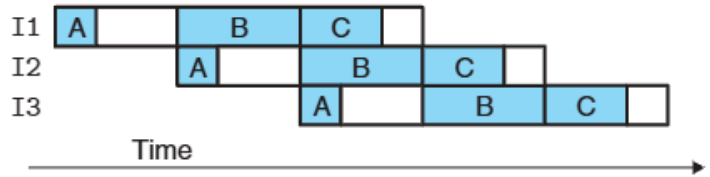

100 ps
20 ps
100 ps
20 ps
100 ps
20 ps
Comb. logic A
Reg
Comb. logic B
Reg
Comb. logic C
Reg
Clock

3. Limitations of Pipelining

Nonuniform Partitioning



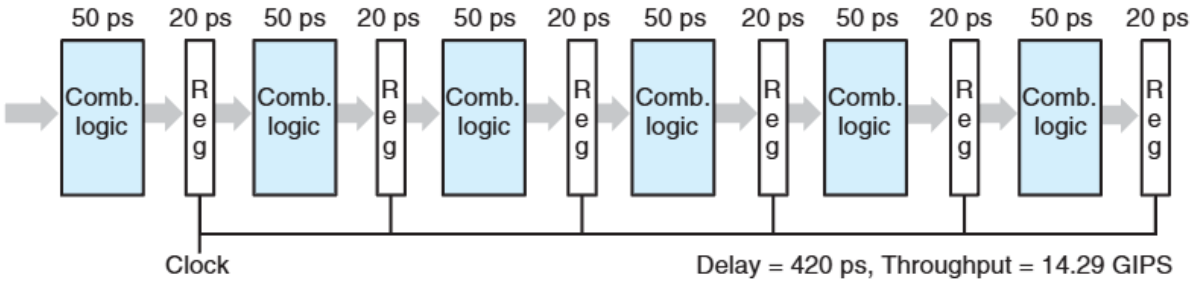
(a) Hardware: Three-stage pipeline, nonuniform stage delays



(b) Pipeline diagram

- A 阶段浪费了 100 ps
  C 阶段浪费了 50 ps
- **慢阶段决定全局时钟周期**
- 处理方法：
  - 划分为尽可能均匀的阶段
  - 优化慢阶段

Diminishing Returns of Deep Pipelining



- 减半了每个 Combinational Logic 的计算用时
- 整体吞吐量只提升了不到 2 倍
- 原因是寄存器有开销（占了 28.6%）

现代处理器采用了非常深的流水线（至少 15 个阶段），以提高时钟频率。处理器架构师需要**把指令执行过程分为低延迟简单步骤**；电路设计者需要**降低寄存器延迟**；芯片设计者需要**确保时钟在整个芯片上同时改变**。

4. Pipelining a System with Feedback

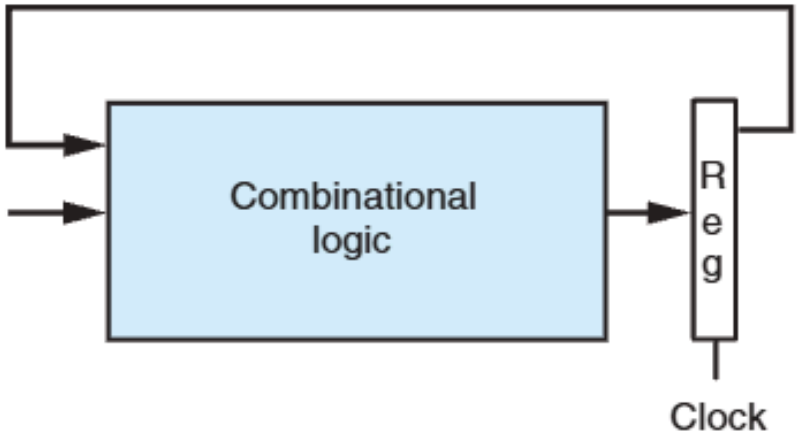
纯计算流水线只需把一个较大的计算拆成多个阶段，但实际的指令可能会读写同一组寄存器、条件码与内存，后续指令可能需要前一条指令生成的结果（Data Dependency，**数据依赖**）：

```
1     irmovq  $50, %rax
2     addq    %rax, %rbx          # %rax 的值依赖第 1 句的结果
3     mrmovq  100(%rbx), %rdx     # %rbx 的值依赖第 2 句的结果
```

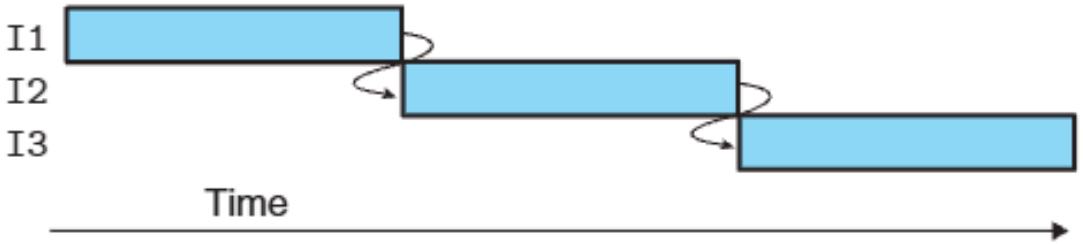
控制语句可能存在**控制依赖**：

```
loop:
    subq    %rdx, %rbx
    jne     targ            # 是否跳转依赖上一语句的结果
    irmovq  $10, %rdx
    jmp     loop
targ:
    halt
```

The Feedback Mechanism of SEQ

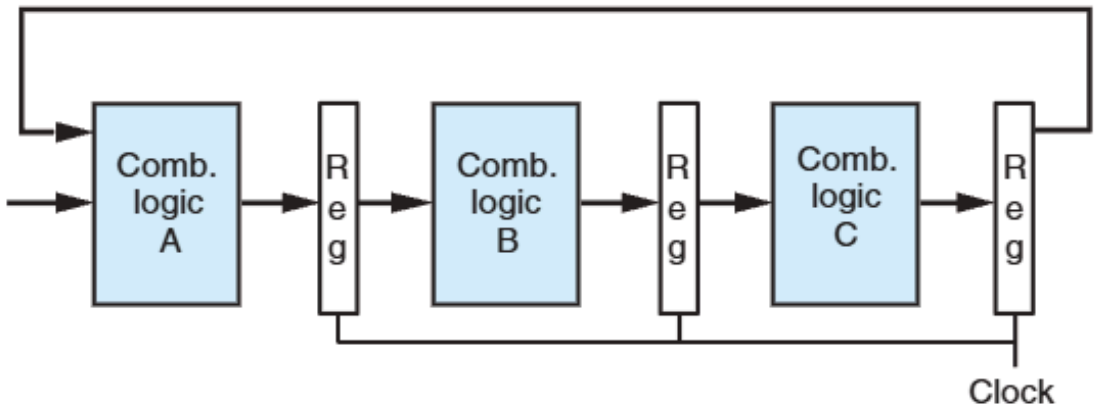


(a) Hardware: Unpipelined with feedback

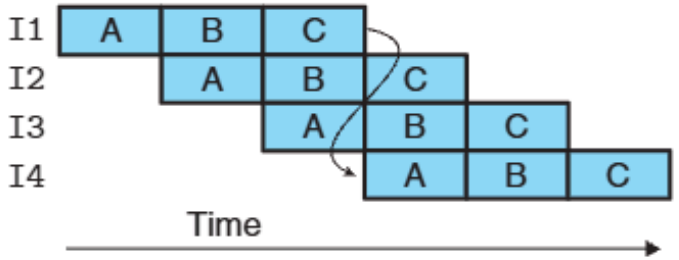


(b) Pipeline diagram

Incorrect Pipeline Feedback Mechanism



(c) Hardware: Three-stage pipeline with feedback



(d) Pipeline diagram

- 如果直接简单地在此基础上把 Combinational Logic 切分，我们就事实上错误地改变了系统的行为（图 (c)(d)）
- 因此，流水线设计的核心问题从“能不能并行”转化为“**在并行发生时，如何仍然表现得像按程序顺序执行**”。后续所有处理机制都围绕这个等价性展开。

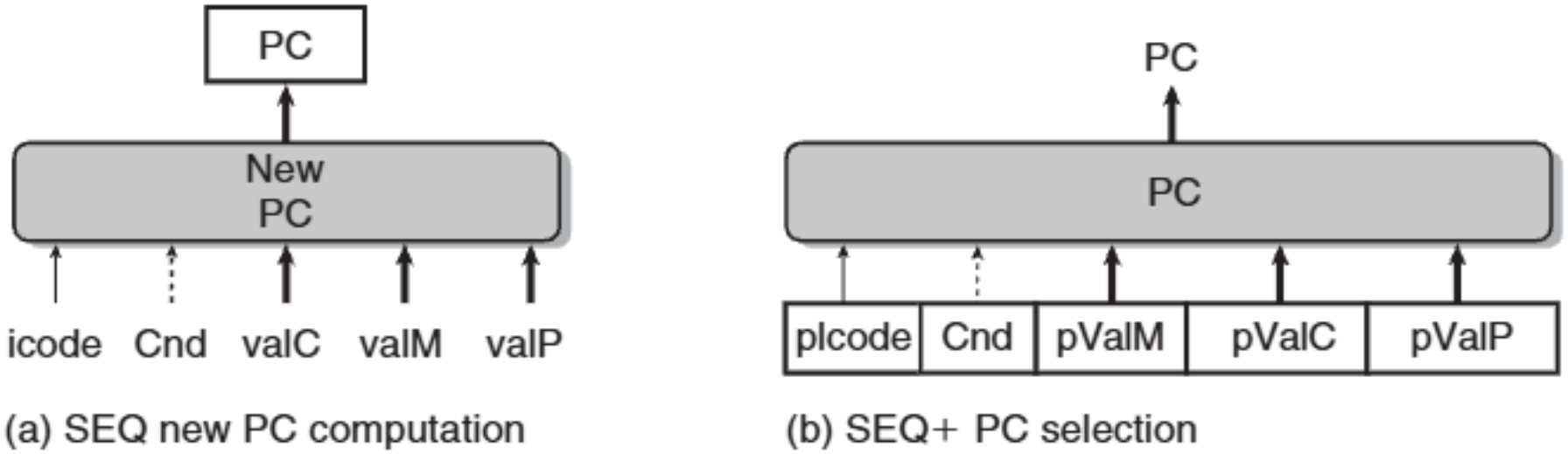
II. Pipelined Y86-64 Implementations

（我们采用教材的讲解顺序，首先从比较 high-level 的抽象视角来理解 PIPE，忽略 HCL 实现；等到对整体框架有大致把握以后，再具体展开 HCL 细节。）

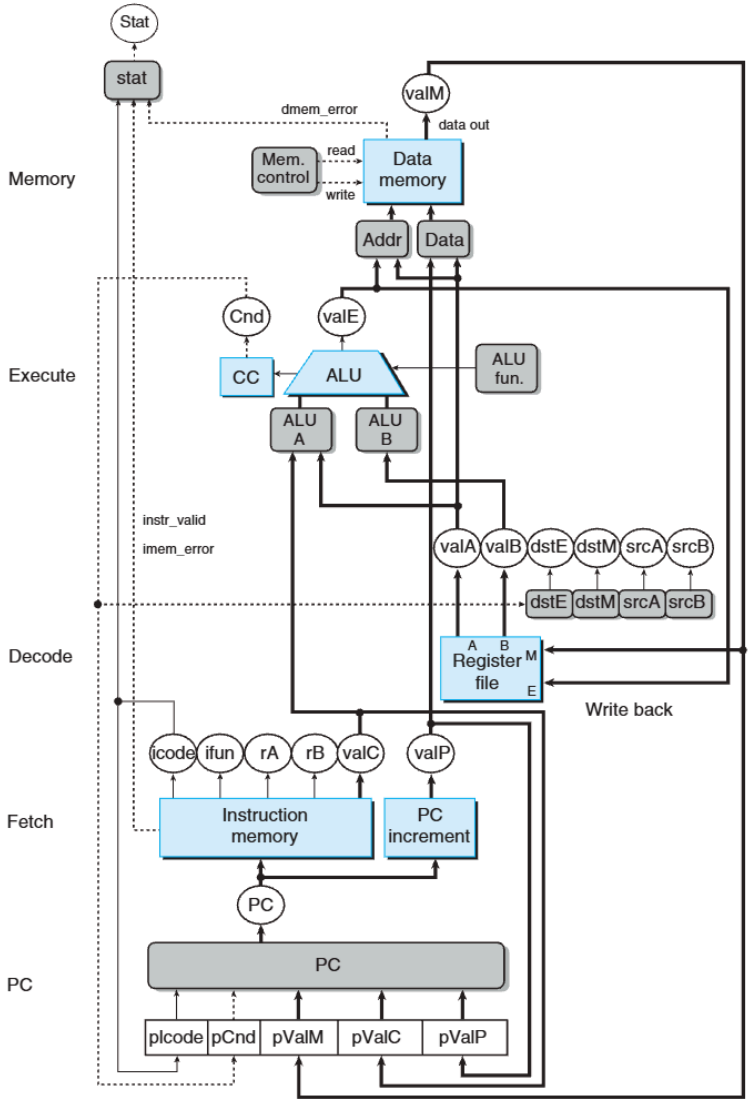
SEQ 中，PC Update 逻辑位于一个周期的末端：当前指令完成后，处理器才决定下一周期从哪里 Fetch。对流水线化而言，Fetch 阶段需要在周期一开始就知道要读取哪个地址。因此我们首先引入 **SEQ+**，把 “下一 PC 的选择” 调整到更适合周期起点的位置。

1. SEQ+: Rearranging the Computation Stages

我们改变 PC 的计算方式，添加新的寄存器：

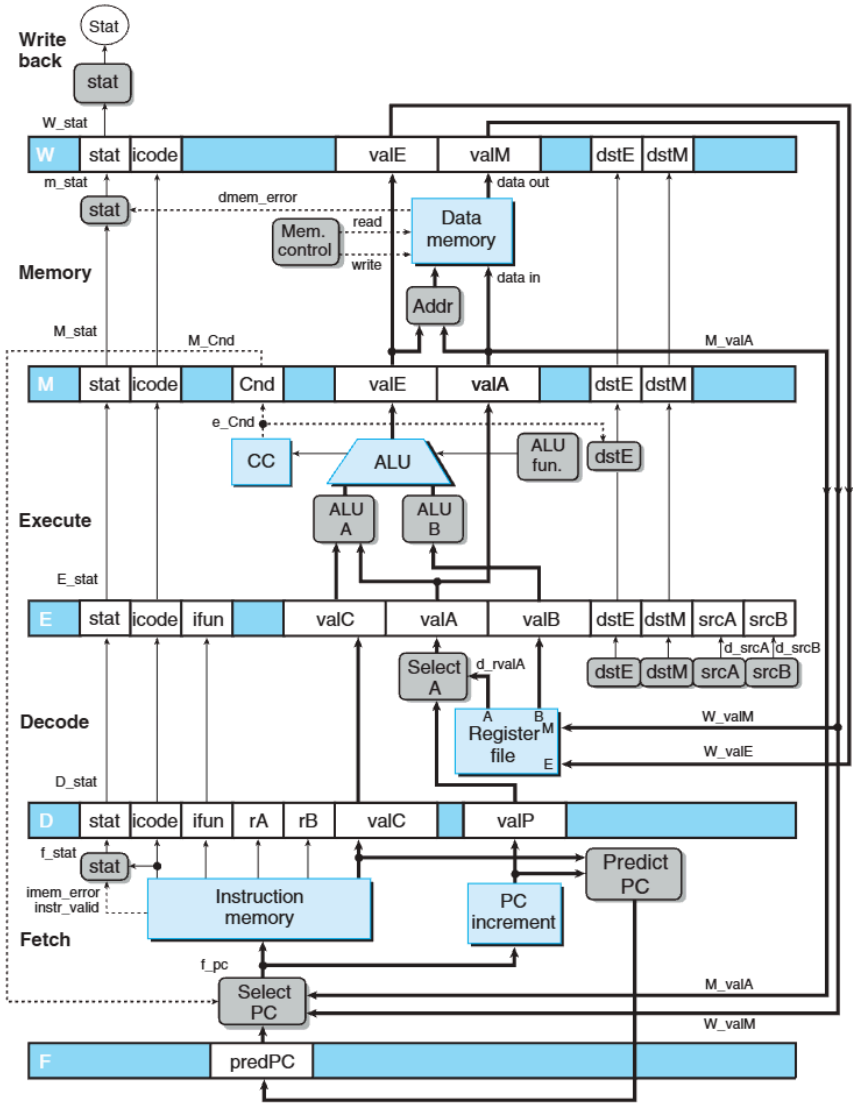


然后把 PC Update 逻辑从指令末尾移到开头：



- 这个变更只是一个把控制时序整理成便于插入流水线寄存器的过渡设计
- 值得注意的是，SEQ+ 的 PC register 被移除了，program counter 是通过新增的寄存器动态计算的

2. Inserting Pipeline Registers $\rightarrow$ PIPE$-$



3. Rearranging and Relabeling Signals

1. 用命名来区分流水线中**同一周期内可能存在的对应于不同指令的多个信号版本**。
   比如说 PIPE− 中的 `Stat` 在同一时刻有 4 个版本：
   - 4 个**寄存器存储的状态码**名字分别为 `D_stat`、`E_stat`、`M_stat`、`W_stat`
   - **各阶段计算得到的状态码**则命名为：`f_stat`、`d_stat`、`e_stat`、`m_stat`

2. Decode 阶段产生的 `dstE` 和 `dstM` 只在抵达 Write Back 阶段时传到 Register file。

3. PIPE− 在 SEQ+ 的基础上移除了 `Data` 控制逻辑，增加了 `Select A` 控制逻辑，来合并 `valP` 和 `valA`，减少了需要传输的状态数量（毕竟只有 `call` 和 `jXX` 指令需要在 Execute 或 Memory 阶段用到 `valP`）。

4. Next PC Prediction

为了实现流水线，某指令 I_1 的 Fetch 阶段完成时我们必须知道下一指令 I_2 的地址，但是如果 I_1 是条件分支语句 / `ret`，那么要得到准确的 I_2 地址还需要几个周期。

- 对于非 `ret` 的语句，PIPE− 采用简单的静态预测策略（这样只有条件跳转可能出错）：

$$\text{The address of the next instruction} = \begin{cases} \text{valC}, & \text{call / jXX}, \\ \text{valP}, & \text{otherwise}. \end{cases}$$

- 对于 `ret`，PIPE− 暂停执行其他指令，直到 `ret` 执行完 Write Back 阶段。

`Predict PC` 控制逻辑用来预测 PC，流水线寄存器 `F : predPC` 用来存储预测的 PC。

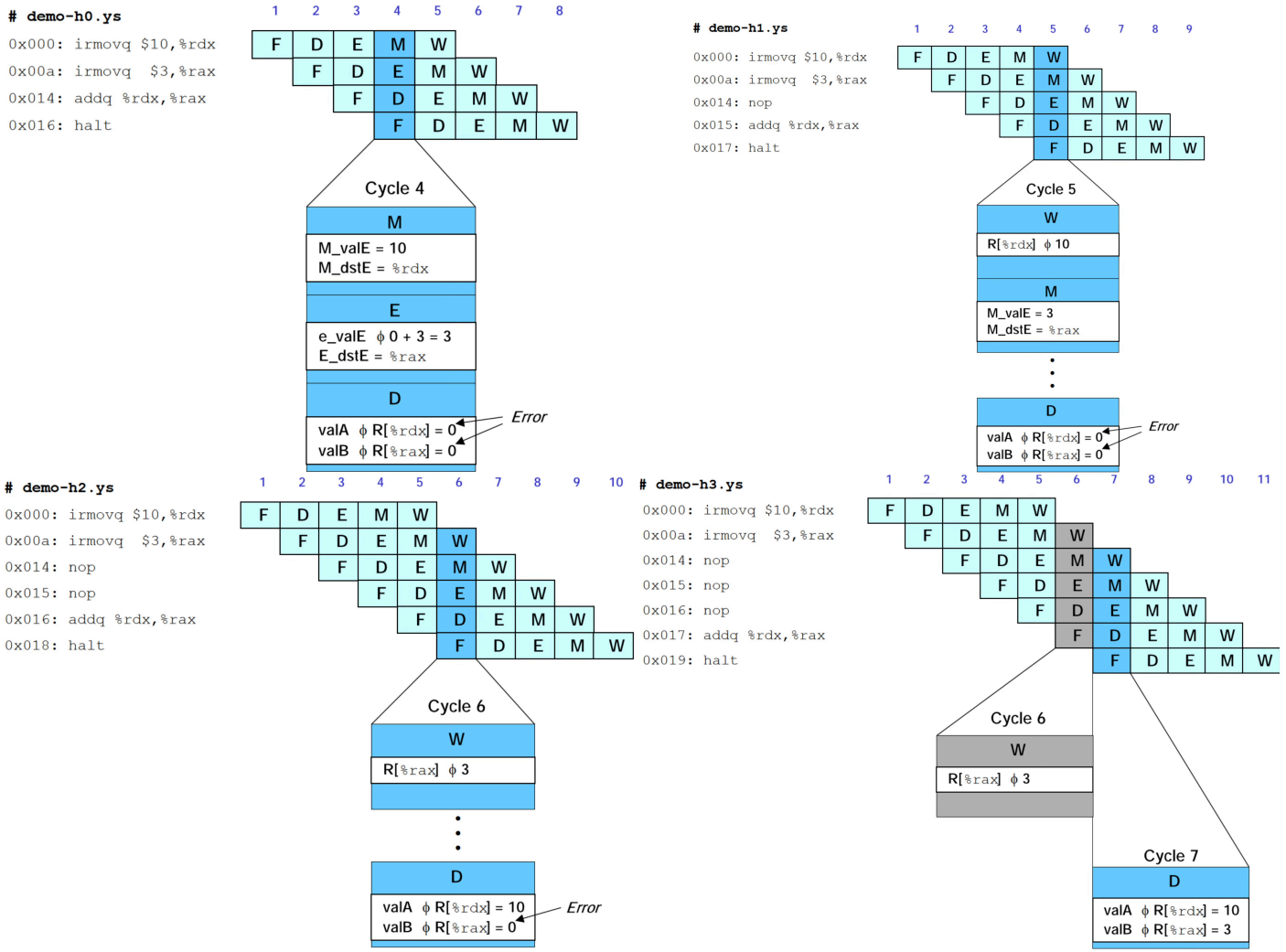
`Select PC` 后续在分支预测错误时在 `predPC`、`M_valA`、`W_valM` 中选择正确地址。

5. Pipeline Hazards

首先考察数据冒险，我们可以穷尽枚举数据冒险的种类：

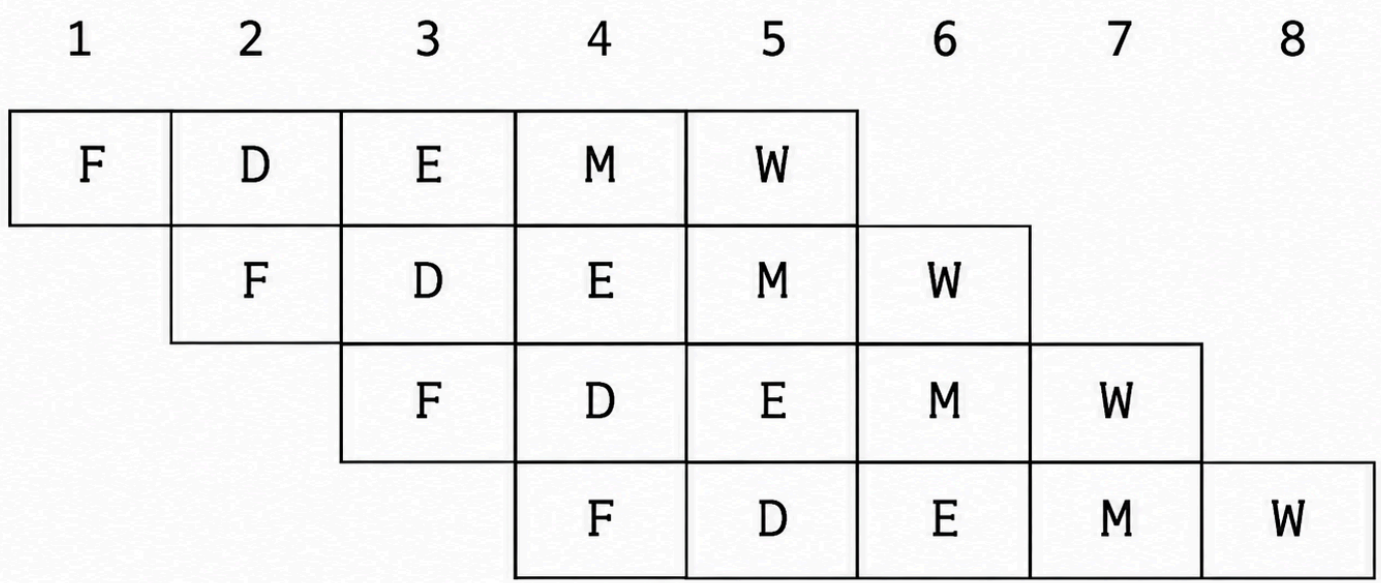
- **Program registers**：Register file 在一个阶段被读取，而在另一个阶段被写入。
- **Program counter**：若 Fetch 阶段逻辑能在取下一条指令前正确预测新PC值，则无冒险。预测错误的分支和 `ret` 指令需特殊处理。
- **Memory**：数据内存的读写不存在内部冲突。但写内存与 Fetch 阶段的读指令可能在代码修改自身的情况下冲突。为简化，我们假设程序不会修改自身，**无冒险**。
- **Condition code register**：由整数运算在 Execute 阶段写入；由条件移动指令（Execute 阶段读）和条件跳转（Memory 阶段读）读取。当条件移动或跳转执行时，之前的整数运算已完成，故**无冒险**。
- **Status register**：流水线中的指令可能影响程序状态。将状态码与每条指令关联，使得处理器能在发生异常时有序停机，需要**异常处理**。

The Simplest Program Register Hazard



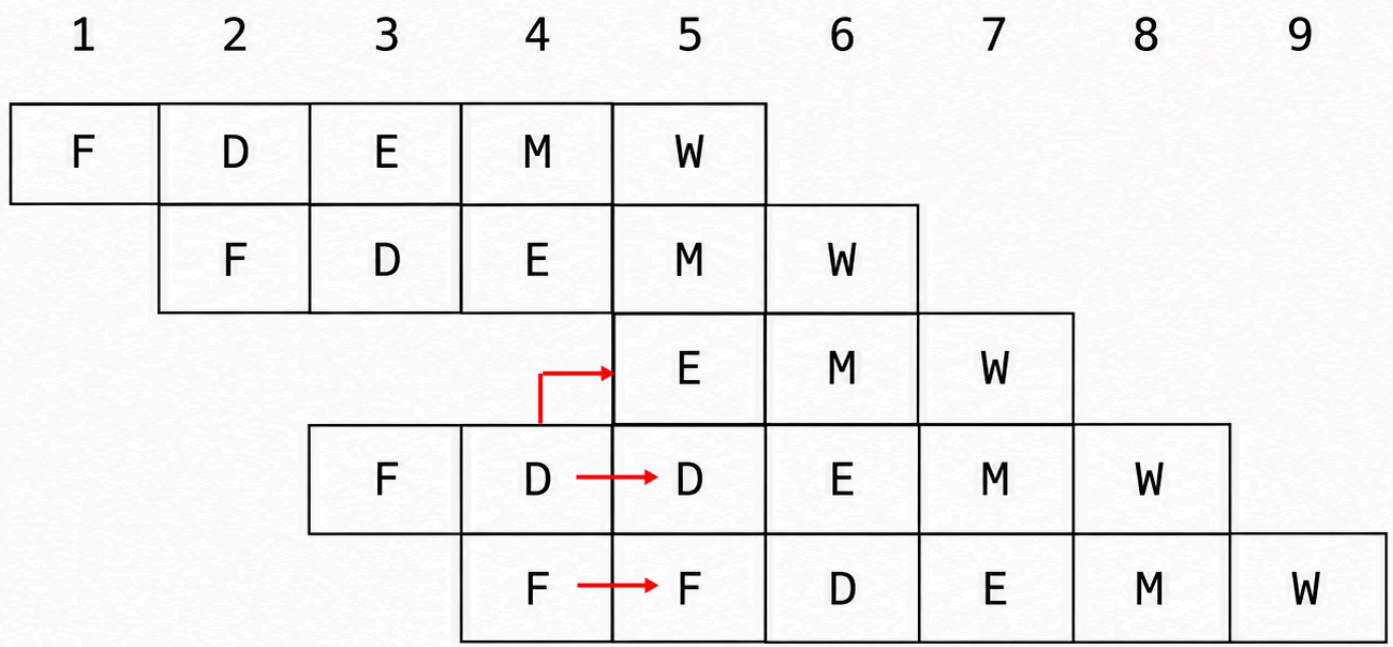
Stalling

```
# prog4
0x000: irmovq $10,%rdx
0x00a: irmovq $3,%rax
0x014: addq %rdx,%rax
0x016: halt
```

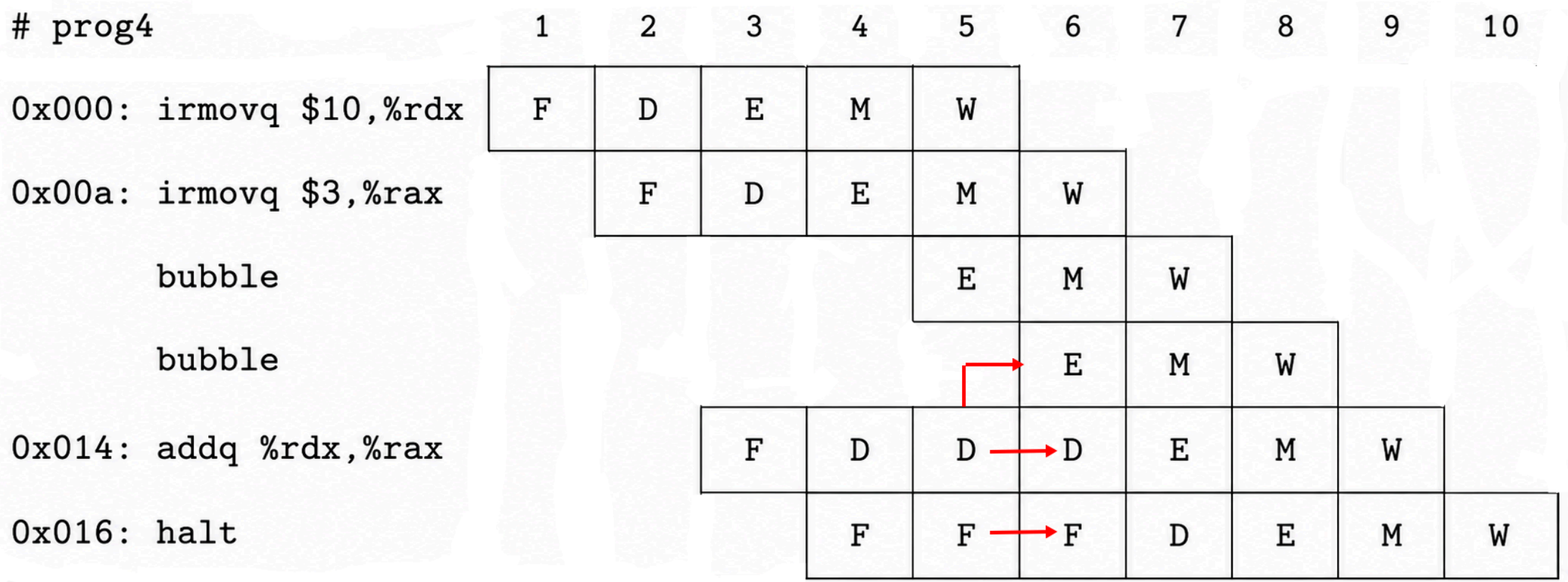


Stalling

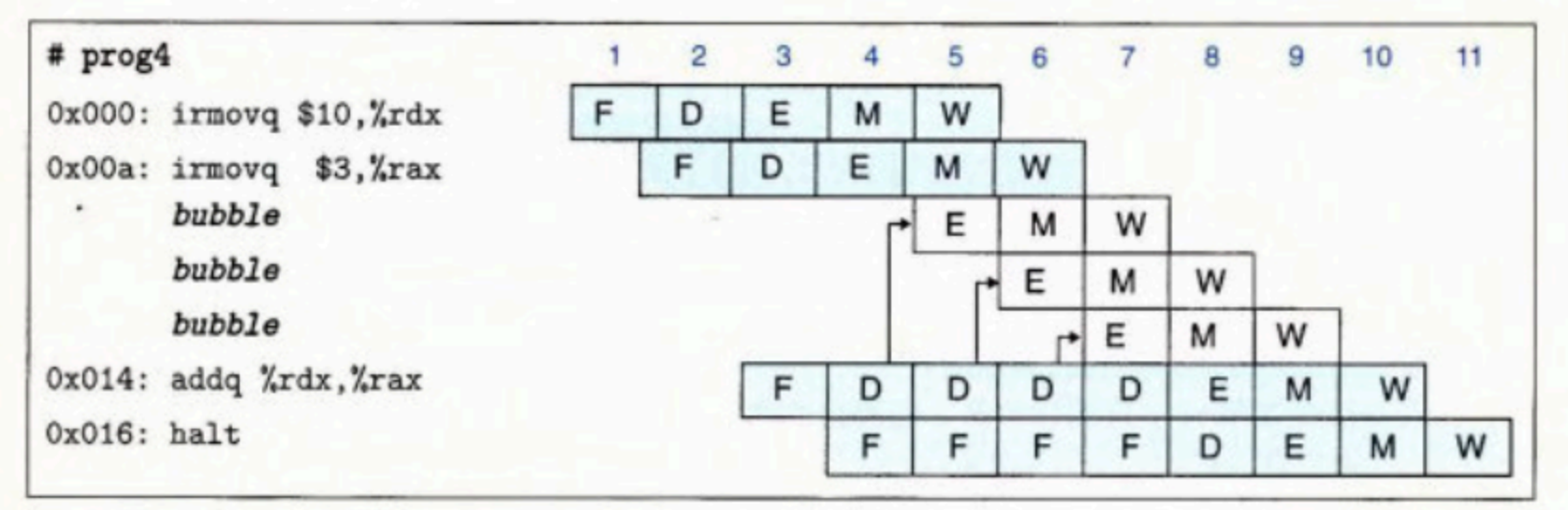
```
# prog4
0x000:  irmovq $10,%rdx
0x00a:  irmovq $3,%rax
        bubble
0x014:  addq %rdx,%rax
0x016:  halt
```



Stalling

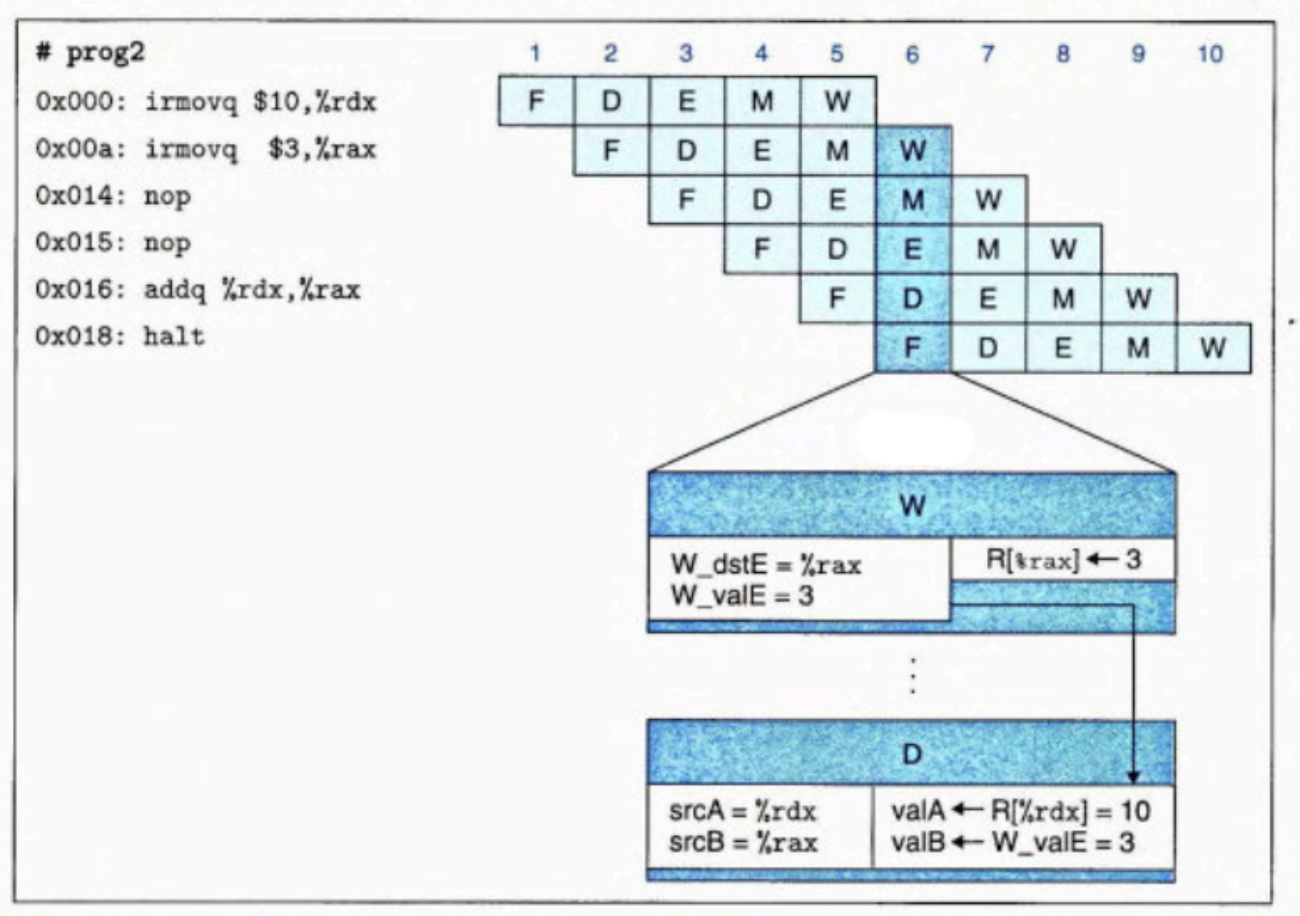

prog4
0x000: irmovq $10,%rdx
0x00a: irmovq $3,%rax
bubble
bubble
0x014: addq %rdx,%rax
0x016: halt
1
2
3
4
5
6
7
8
9
10
F D E M W
F D E M W
E M W
E M W
F D D D E M W
F F F D E M W

最终结果：



暂停的缺点在于会大幅降低吞吐量。

Forwarding



可利用的转发源：

- E stage: `e_valE`
- M stage: `m_valM`, `M_valE`
- W stage: `W_valM`, `W_valE`

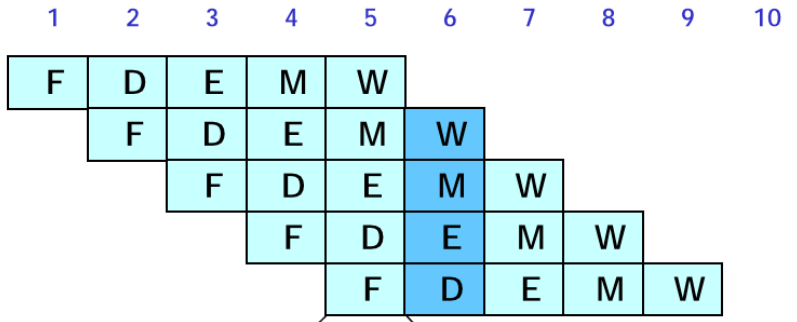
转发目的地：

- `valA`
- `valB`

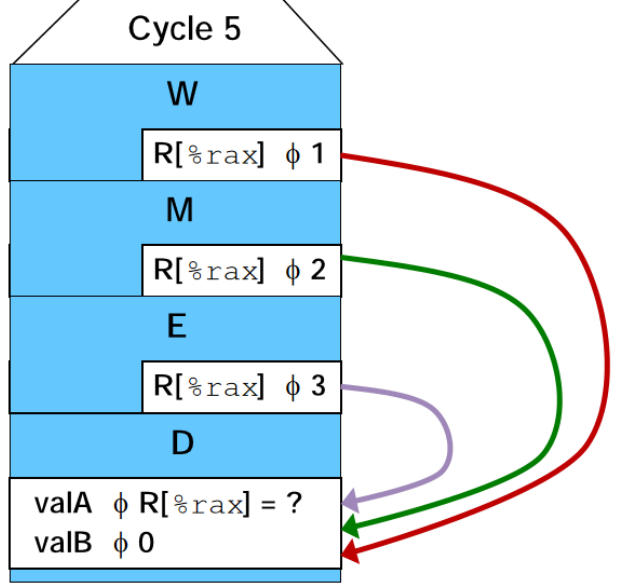
数据转发需要增加额外的数据连接和控制逻辑。

Decode stage 的逻辑比对 `srcA`、`srcB` 以及和写回值绑定的目的寄存器 IDs 来检测是否需要数据转发。有时可能有多个写回值绑定同一寄存器，那么我们就要区分优先级：

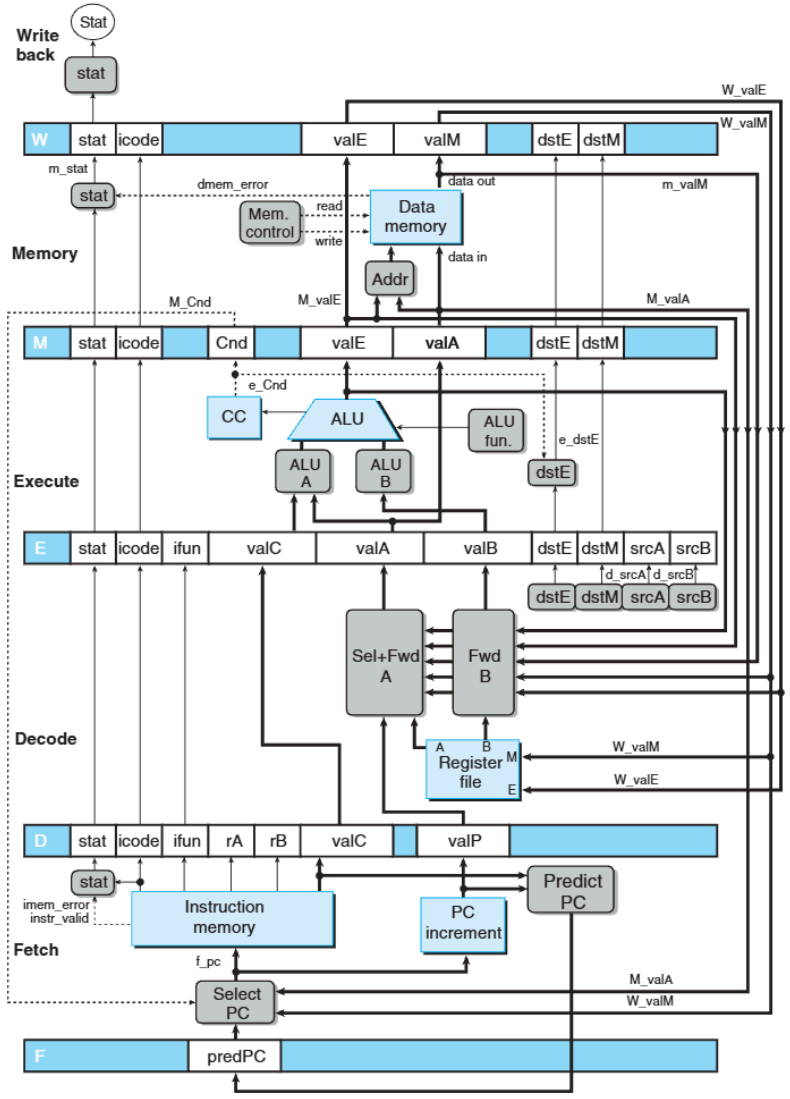
```
# demo-priority.ys
0x000: irmovq $1, %rax
0x00a: irmovq $2, %rax
0x014: irmovq $3, %rax
0x01e: rrmovq %rax, %rdx
0x020: halt
```



- **Multiple Forwarding Choices**
  - **Use matching value from earliest pipeline stage**

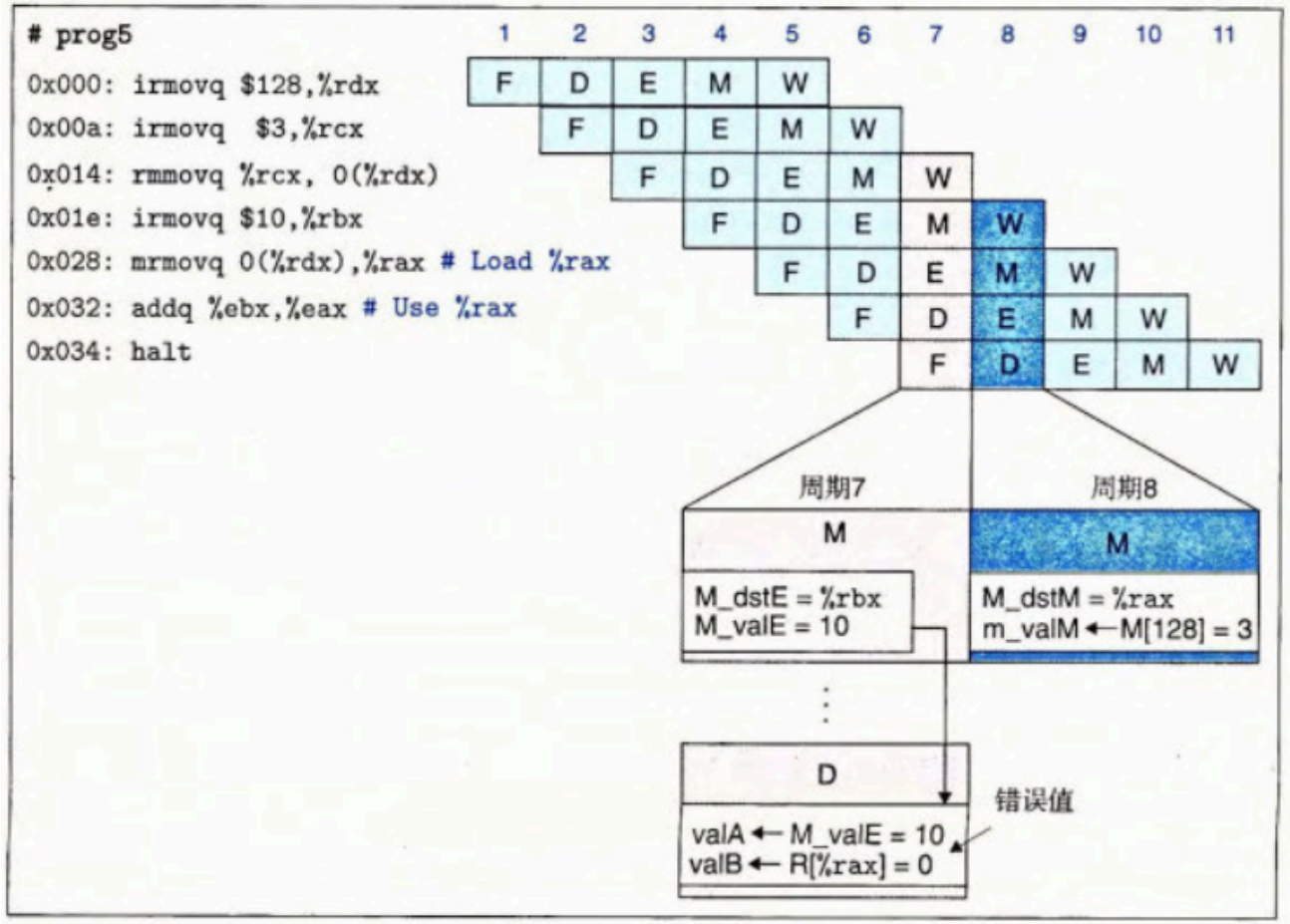


PIPE − + Forwarding → PIPE

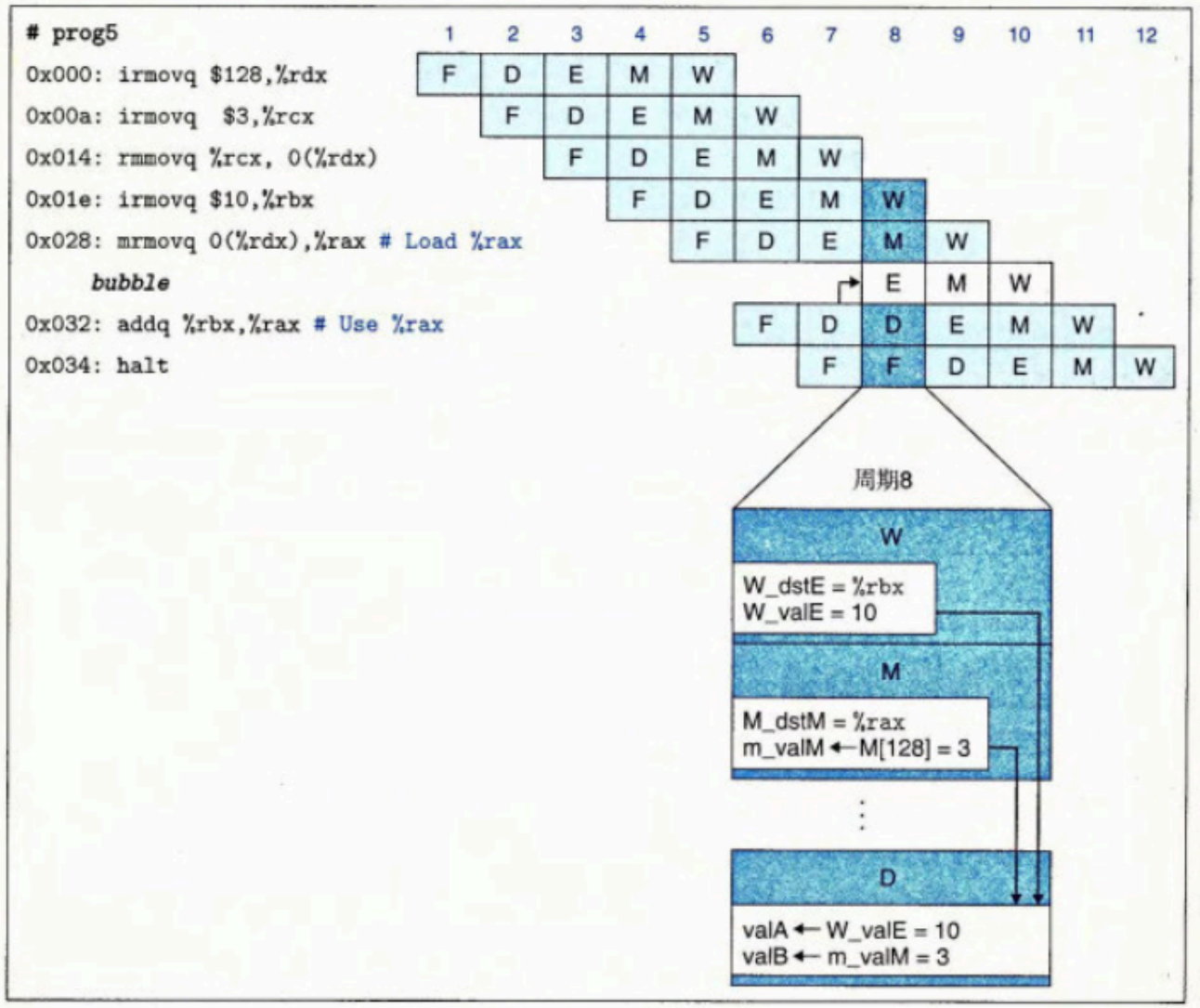


Load/Use Data Hazards

一种数据转发无法解决的数据冒险：加载 / 使用数据冒险，本质是 Memory 阶段靠后。

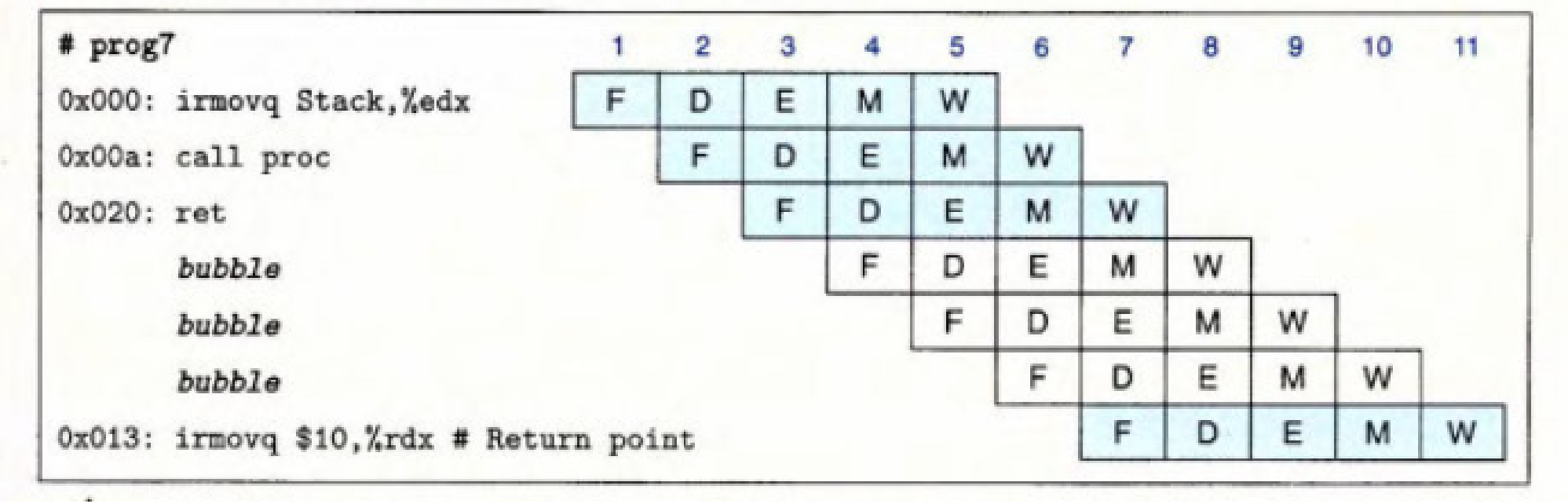


解决方案就是结合暂停和转发（*Load interlock*，加载互锁）：



Control Hazards

`ret` 的处理很简单，单纯暂停即可：

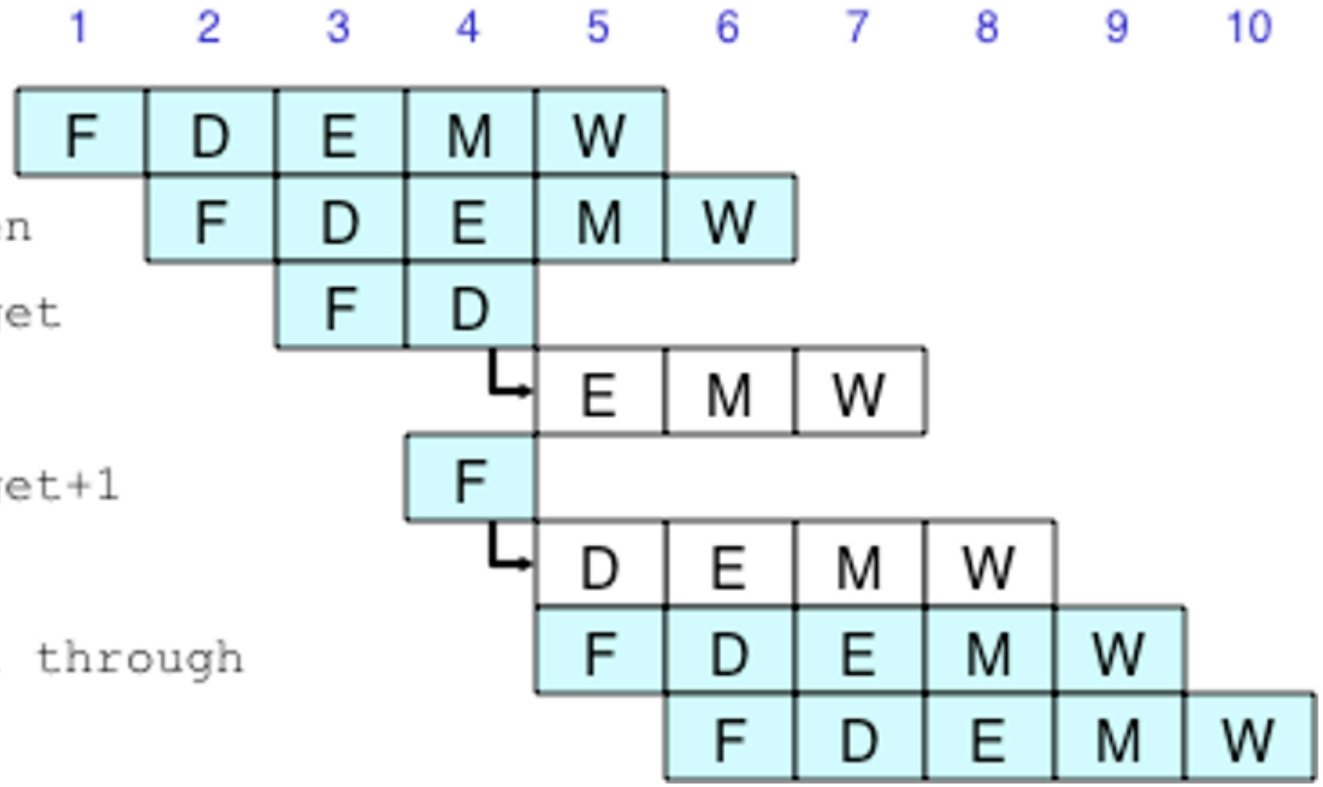


处理错误的分支预测：

demo-j.ys

```
0x000:      xorq %rax,%rax
0x002:      jne  t                  # Not taken
0x00b:      irmovq $1, %rax         # Fall through
0x015:      nop
0x016:      nop
0x017:      nop
0x018:      halt
0x019: t:   irmovq $3, %rdx         # Target
0x023:      irmovq $4, %rcx         # Should not execute
0x02d:      irmovq $5, %rdx         # Should not execute
```

```
# demo-j.ys
0x000: xorq %rax,%rax
0x002: jne target # Not taken
0x016: irmovq $2,%rdx # Target
       bubble
0x020: irmovq $3,%rbx # Target+1
       bubble
0x00b: irmovq $1,%rax # Fall through
0x015: halt
```



当一条 `jXX` 进入 E 阶段时，处理器可根据当前条件码算出 `e_Cnd`。若 `e_Cnd` 为真，预测正确，流水线继续；若 `e_Cnd` 为假，说明 Fetch 阶段错误地去取了目标路径上的指令，PIPE 会通过注入气泡清除错误的指令，上述例子中的代价是浪费了 2 个周期。

6. Exception Handling

Y86-64 架构中可能触发异常的 3 种情况：

1. `halt`
2. 某条指令的 `icode:ifun` 组合无效
3. 地址访问异常（Fetch / Memory）

3 种微妙的情况：

1. 多条指令同时触发异常：**取最前的一条**
2. 一条指令触发了异常，但是之后发现是分支预测错误被撤销了
3. 一条指令触发异常时，随后的流水线指令已经修改了程序员可见状态

第一种、第二种情况由于 `stat` 寄存器的流水线顺序轻松解决。
第三种情况：流水线控制逻辑需要在检测到异常时禁止后续指令写入 CC / Memory。

7. PIPE Stage Implementations

接下来我们介绍各阶段的详细 HCL 实现。

PC Selection and Fetch Stage

↓

Decode and Write-Back Stages

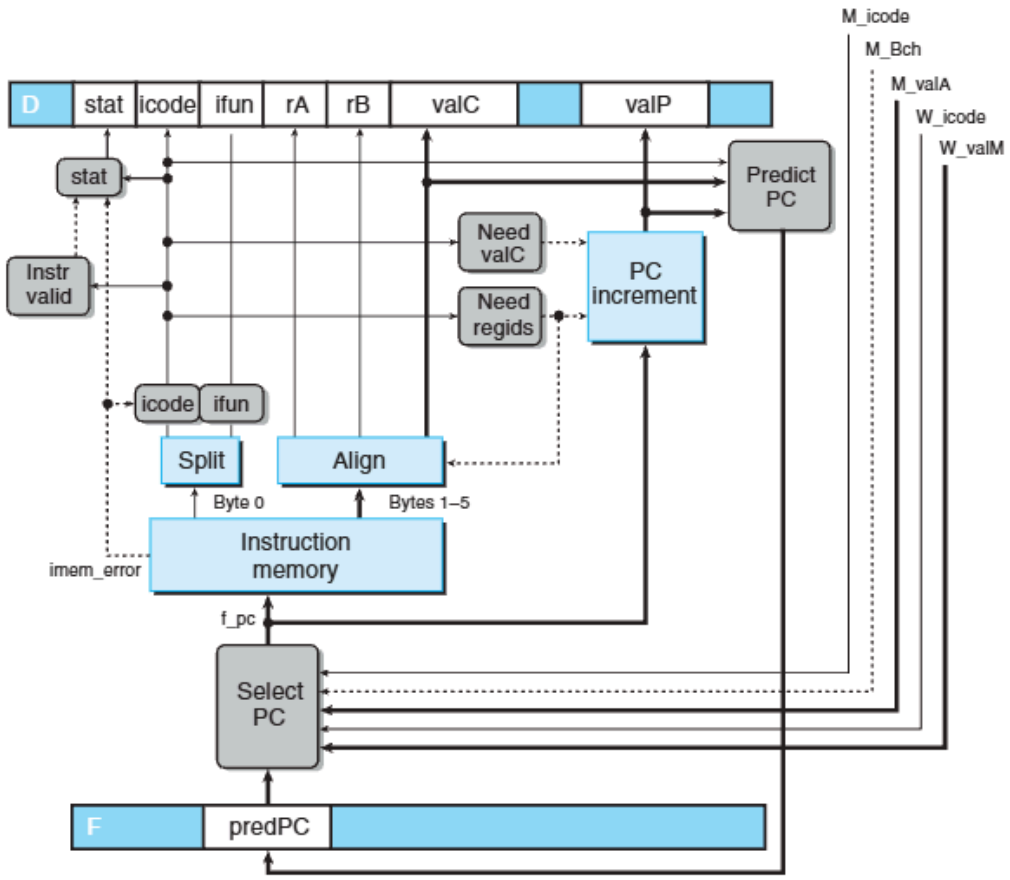
↓

Execute Stage

↓

Memory Stage

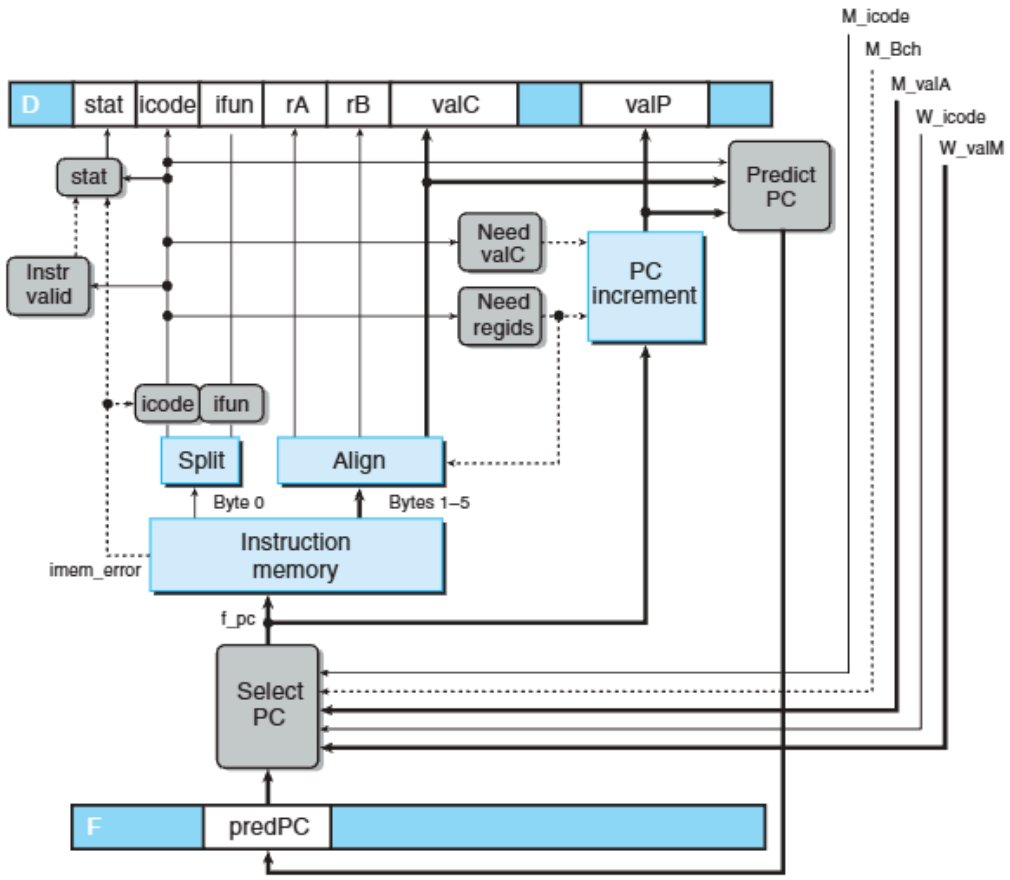
PC Selection and Fetch Stage




```
word f_pc = [
    # Mispredicted branch. Fetch at incremented PC
    M_icode = IXX && !M_Cnd : M_valA;
    # Completion of RET instruction
    W_Icode == IRET : W_valM;
    # Default: Use predicted value of PC
    1 : F_predPC;
];
```

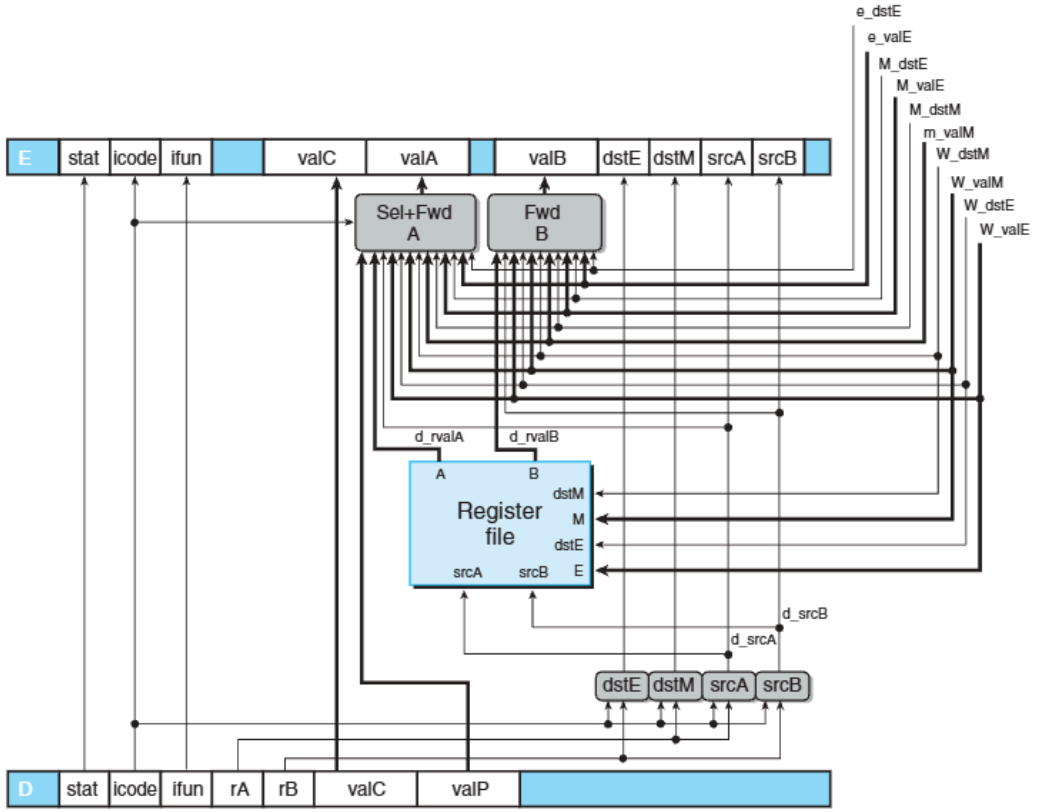
```
word f_predPC = [
    f_icode in { IJXX, ICALL } : f_valC;
    1 : f_valP;
];
```

PC Selection and Fetch Stage




```
word f_stat = [
    imem_error : SADR;
    !instr_valid : SINS;
    f_icode == IHALT : SHLT;
    1 : SAOK;
];
```

Decode and Write-Back Stages

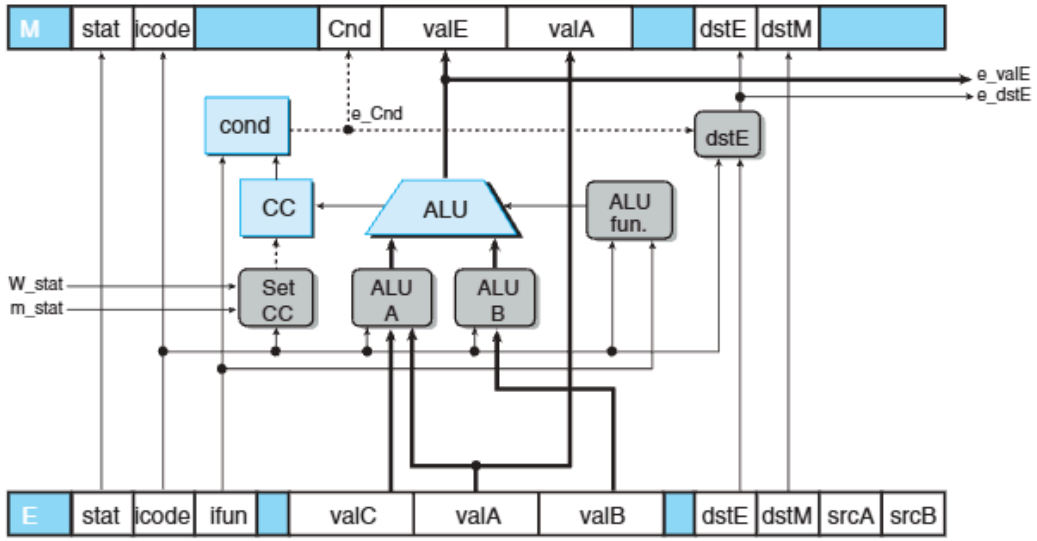



```
word d_dstE = [
    D_icode in { IRRMOVQ, IIRMOVQ, IOPQ } : D_rB;
    D_icode in { IPUSHQ, IPOPQ, ICAL, IRET } : RRSP;
    1 : RNONE;  # Don't write any register
];
```

```
word d_valA = [
    D_icode in { ICALL, IJXX } : D_valP;  # Use incremented PC
    d_srcA == e_dstE : e_valE;    # Forward valE from execute
    d_srcA == M_dstM : m_valM;    # Forward valM from memory
    d_srcA == M_dstE : M_valE;    # Forward valE from memory
    d_srcA == W_dstM : W_valM;    # Forward valM from write back
    d_srcA == W_dstE : W_valE;    # Forward valE from write back
    1 : d_rvalA;  # Use value read from register file
];
```

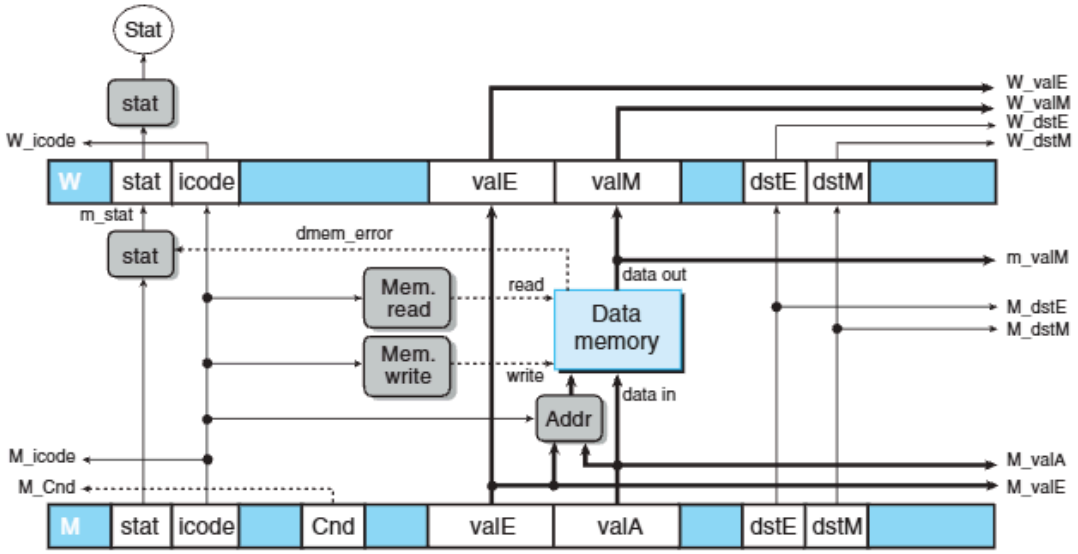
```
## The Write-back stage final Stat
word Stat = [
    W_stat == SBUB : SAOK;
    1 : W_stat;
];
```

Execute Stage



- Set CC 控制逻辑以 `W_stat` 、 `m_stat` 作为输入，决定是否修改 CC

Memory Stage



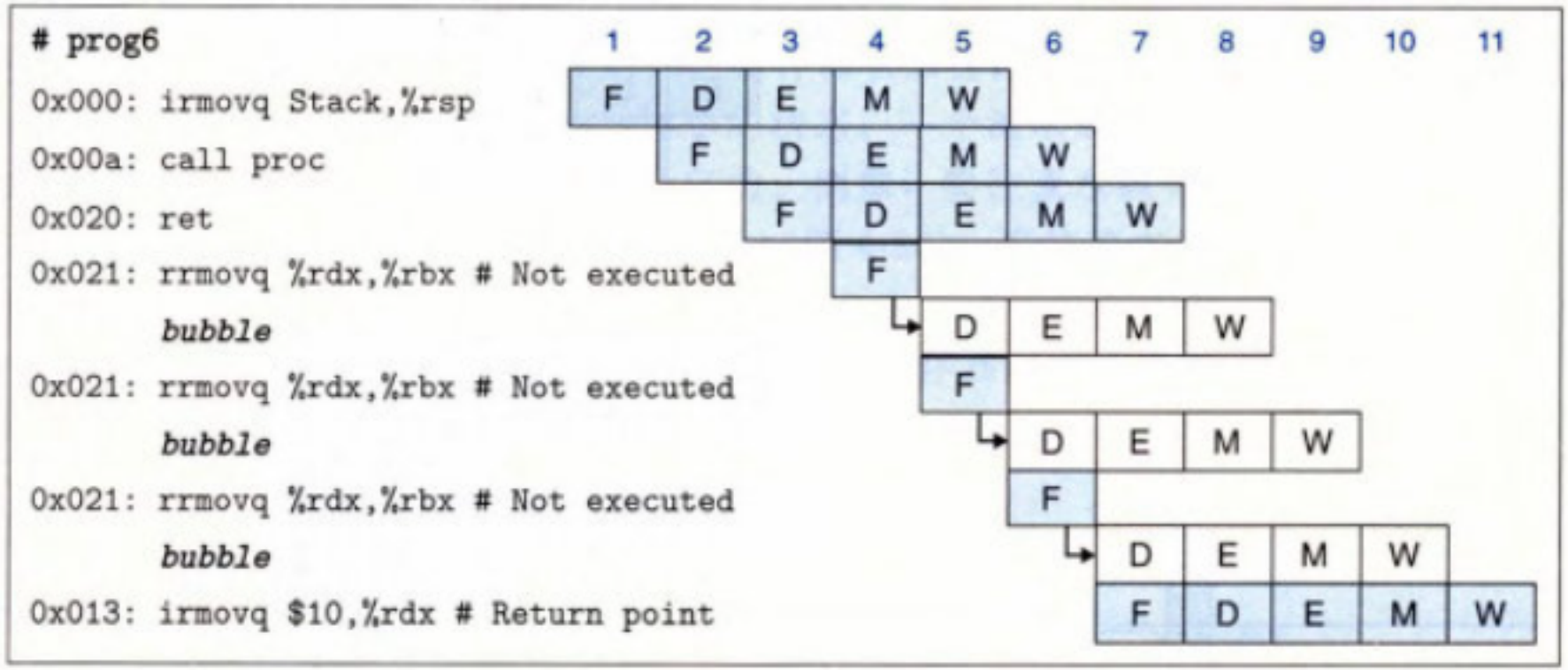
- Mem. data 控制逻辑被移除
- 其余控制逻辑基本与 SEQ 一致

```
word m_stat = [
    dmem_error : SADR;
    1 : M_stat;
];
```

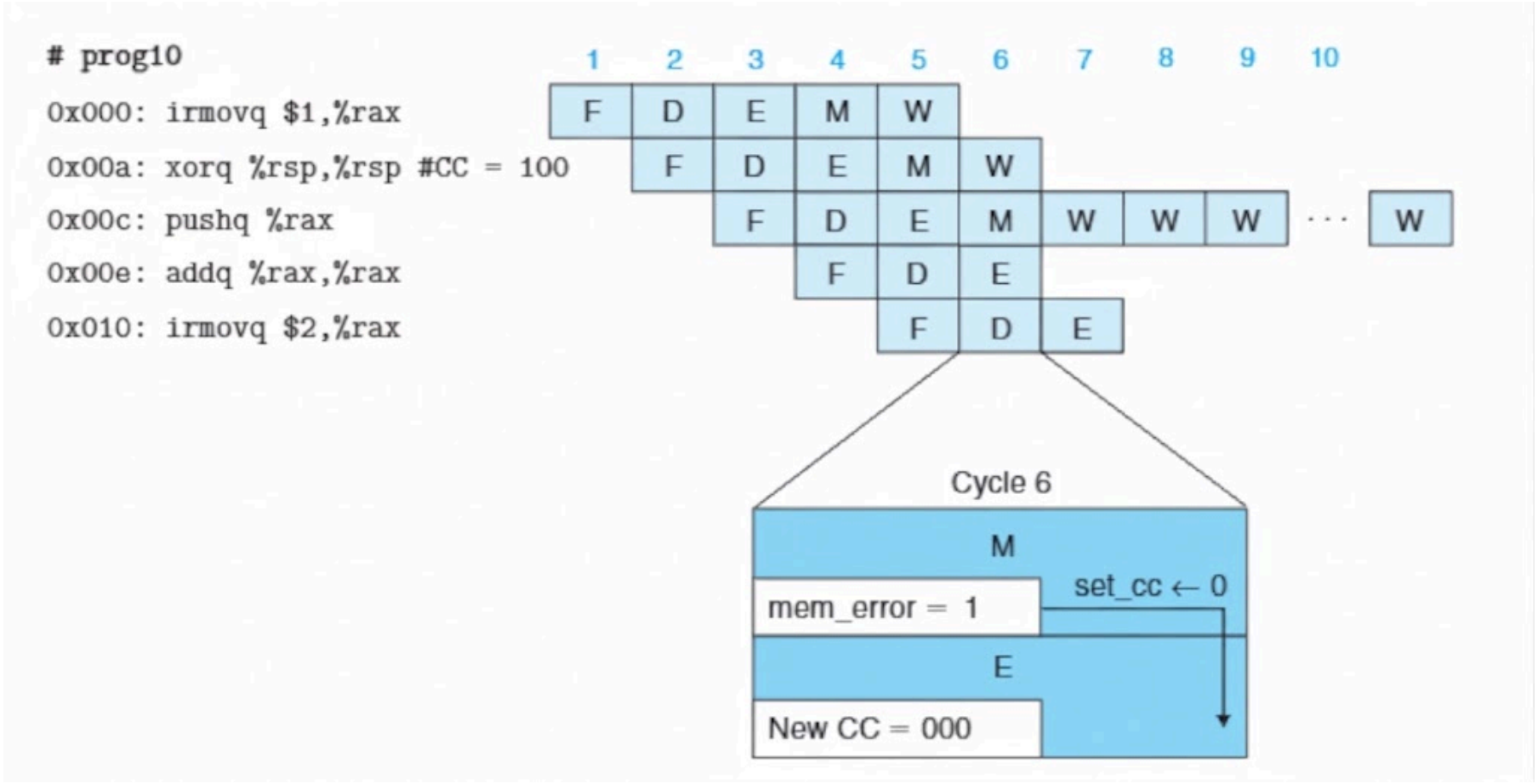
8. Pipeline Control Logic

- *Load/use hazards*
  - `mrmovq` / `popq`
  - detect the hazard condition
  - keep pipeline registers F & D fixed
  - inject a bubble into the E stage
- *Processing* `ret`
- *Mispredicted branches*
  - when `jXX` reaches the E stage
  - inject bubbles into the D & E stages / fetch the correst Ins. on the next cycle
- *Exceptions*

ret


prog6
0x000: irmovq Stack,%rsp
0x00a: call proc
0x020: ret
0x021: rrmovq %rdx,%rbx # Not executed
bubble
0x021: rrmovq %rdx,%rbx # Not executed
bubble
0x021: rrmovq %rdx,%rbx # Not executed
bubble
0x013: irmovq $10,%rdx # Return point
1 2 3 4 5 6 7 8 9 10 11
F D E M W
F D E M W
F D E M W
F
D E M W
F
D E M W
F
D E M W
F D E M W

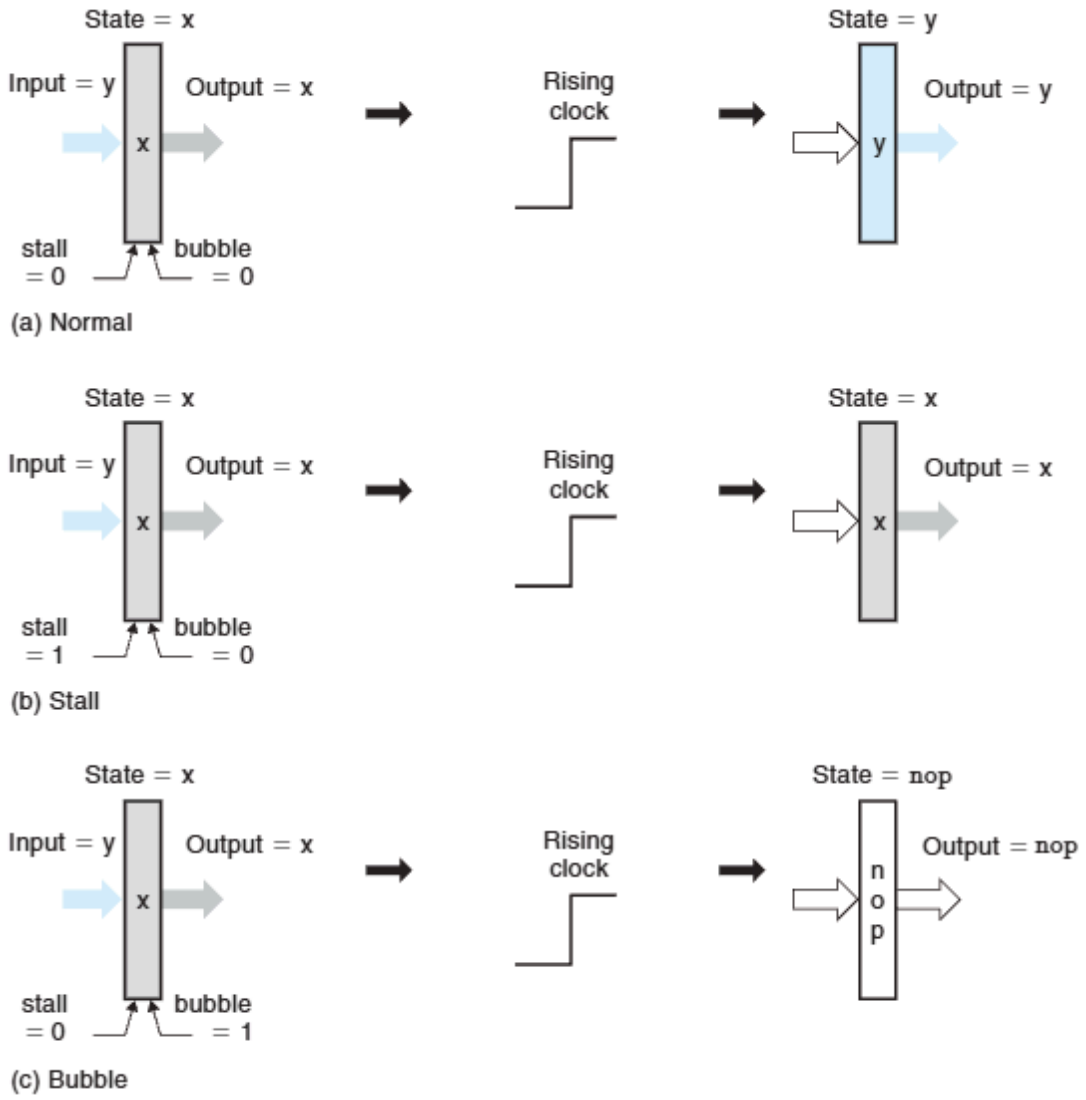
Exception Handling



Detect

Condition	Trigger
Processing `ret`	IRET ∈ {D_icode, E_icode, M_icode}
Load/use hazard	E_icode ∈ {IMRMOVQ, IPOPQ} && E_dstM ∈ {d_srcA, d_srcB}
Mispredicted branch	E_icode = IJXX && ! e_Cnd
Exception	m_stat ∈ {SADR, SINS, SHLT} | | W_stat ∈ {SADR, SINS, SHLT}

Pipeline Control Mechanisms

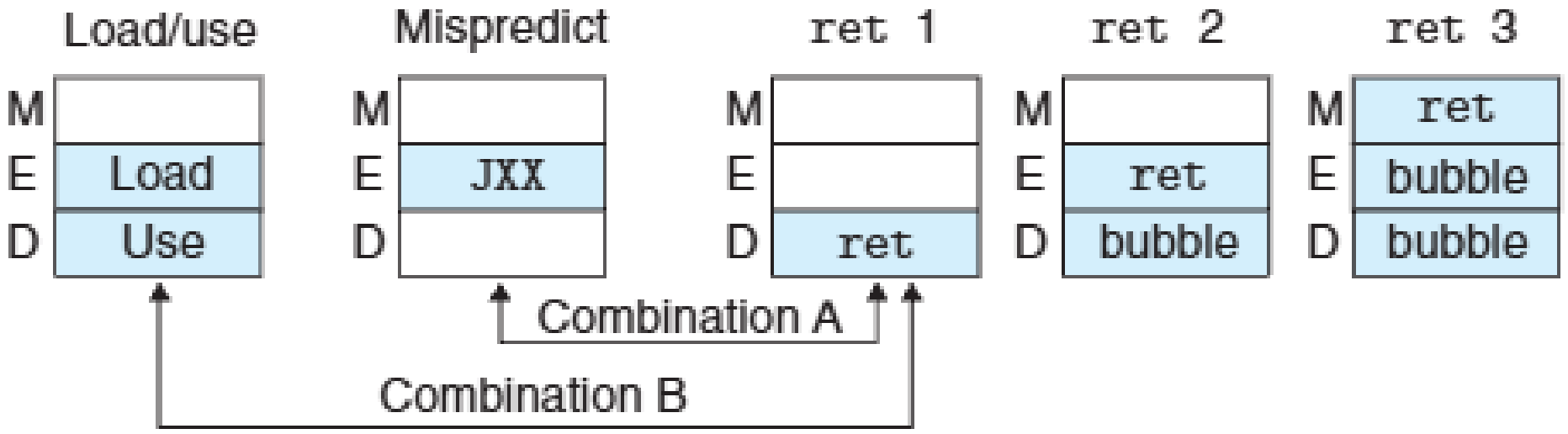


Condition	Pipeline register				
	F	D	E	M	W
Processing `ret`	stall	bubble	normal	normal	normal
Load/use hazard	stall	stall	bubble	normal	normal
Mispredicted branch	normal	bubble	bubble	normal	normal

Combinations of Control Conditions

目前我们的设计只能处理单个特殊情况发生的情况，接下来需要处理多个特殊情况同时发生的情况。

首先，异常同时发生时的处理机制已经完善了，不用再考虑，剩余可能的情况：

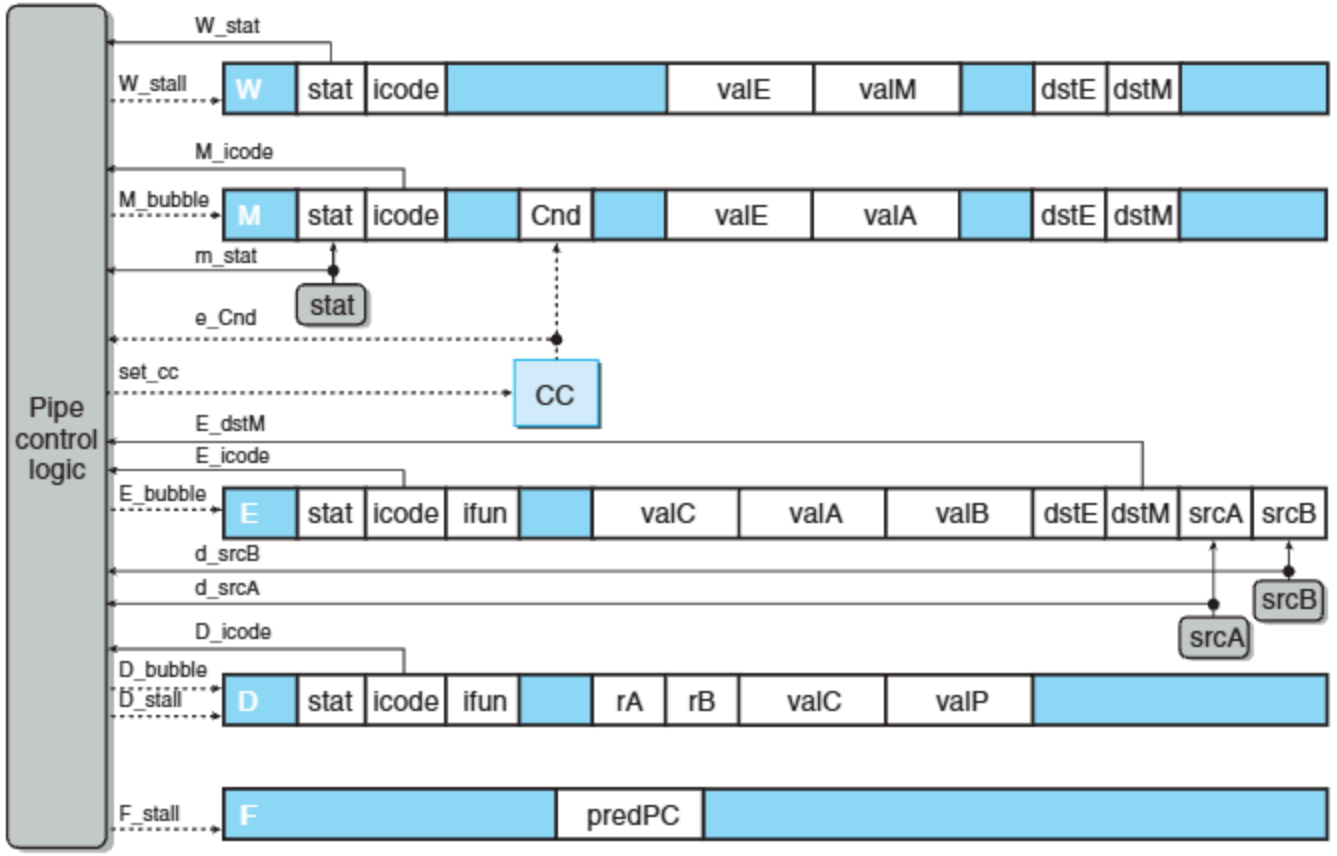


Solution

Condition	Pipeline register F	D	E	M	W
Processing `ret`	stall	bubble	normal	normal	normal
Mispredicted branch	normal	bubble	bubble	normal	normal
Combination	stall	bubble	bubble	normal	normal

Condition	Pipeline register F	D	E	M	W
Processing `ret`	stall	bubble	normal	normal	normal
Load/use hazard	stall	stall	bubble	normal	normal
Combination	stall	bubble+stall	bubble	normal	normal
Desired	stall	stall	bubble	normal	normal

Control Logic Implementation



```
bool F_stall =
    # Conditions for a load/use hazard
    E_icode in { IMRMOVQ, IPOPQ } &&
     E_dstM in { d_srcA, d_srcB } ||
    # Stalling at fetch while ret passes through pipeline
    IRET in { D_icode, E_icode, M_icode };
```

```
bool D_bubble =
    # Mispredicted branch
    (E_icode == IJXX && !e_Cnd) ||
    # Stalling at fetch while ret passes through pipeline
    # but not condition for a load/use hazard
    !(E_icode in  { IMRMOVQ, IPOPQ } && E_dstM in { d_srcA, d_srcB }) &&
      IRET in { D_icode, E_icode, M_icode };
```

```
## Should the condition codes be updated?
bool set_cc = E_icode == IOPQ &&
    # State changes only during normal operation
    !m_stat in { SADR, SINS, SHLT } && !W_stat in { SADR, SINS, SHLT };
```

Exception Handling

```
# Start injecting bubbles as soon as exception passes through memory stage
bool M_bubble = m_stat in { SADR, SINS, SHLT } || W_stat in { SADR, SINS, SHLT };
```

```
bool W_stall = W_stat in { SADR, SINS, SHLT };
```

9. Performance Analysis

CPI (Cycles Per Instruction)

假设异常不发生，那么 CPI 的计算公式为

$$\mathrm{CPI} = \frac{C_i + C_b}{C_i} = 1.0 + \frac{C_b}{C_i}$$

$$= 1.0 + lp + mp + rp$$

假设 Load instructions 占所有指令的 25%，且其中有 20% 会触发加载/使用冒险；条件分支占所有指令的 20%，且其中有 40% 预测错误；返回语句占所有指令的 2%：

计算可得 $\mathrm{CPI} = ?$

10. Unfinished Business

- PIPE 对多周期指令的支持不足：它的控制逻辑默认指令按固定节拍推进，主要面向简单的整数指令；若加入乘法、除法、浮点运算等多周期功能单元，就需要额外的忙/闲状态、可变延迟暂停和更复杂的冒险检测，否则要么只能等待功能单元完成而拉长所有指令的延迟，要么需要大量停顿，导致流水线效率下降、控制复杂度显著上升。
- PIPE 与真实内存系统的接口过于理想化：它假设取指和访存都能在单周期内完成，没有完整处理 cache 缺失、非对齐访问、存储转发、内存映射 I/O 以及访存异常等带来的可变延迟和精确异常问题；一旦内存访问需要多个周期或失败，流水线就必须停顿、冲刷并保证异常精确性，而 PIPE 的简化模型并未提供这些机制。
- 现代处理器：分支预测与指令预取、超标量多发射 + 寄存器重命名 + 乱序执行、重排序缓冲按序提交、多级 Cache / 存储缓冲与预取、同时多线程/多核。